

AZ500 Lab 8-11 Labs 08-11:
Azure Monitor, Microsoft
Defender for cloud, Enable Just-
In Time Access in VMs,
Microsoft Sentinel

RENE OLUOCH CS-CNS09-25030

Table of Contents

Introduction.....	2
Lab 8 Azure Monitor.....	2
Logging in as admin and downloading relevant files	Error! Bookmark not defined.
Exercise 1: Deploying an Azure virtual machine	2
Task 1: Deploying an Azure virtual machine.....	2
Exercise 2: Creating a Log Analytics Workspace	6
Task 1: Creating a Log Analytics Workspace	6
Exercise 3: Creating an Azure Storage Account	8
Task 1: Creating an Azure Storage Account	8
Exercise 4: Creating a Data Collection Rule	10
Task 1: Creating a Data Collection Rule.....	10
Lab 09: Configuring Microsoft Defender for Cloud Enhanced Security Features for Servers	14
Exercise 1: Configuring Microsoft Defender for Cloud Enhanced Security Features for Servers	14
Lab 10: Enabling Just-In-Time Access on VMs	17
Exercise 1: Enabling JIT on your VMs from Azure Virtual Machines.....	17
Exercise 2: Requesting Access to a JIT-Enabled VM from the Azure Virtual Machine's Connect Page	19
Lab 11: Microsoft Sentinel	21
Exercise 1: Onboarding Microsoft Sentinel.....	21
Exercise 2: Connecting Azure Activity to Sentinel.....	22
Exercise 3: Creating a rule that uses the Azure Activity data connector	26
Task 4: Creating a playbook	26
Task 5: Creating a custom alert and configure a playbook as an automated response	28
Task 6: Invoke an incident and review the associated actions.....	30
Conclusion	35

Introduction

In these labs, I gained hands-on experience with a range of advanced Azure security capabilities focused on protecting virtual machines and enhancing visibility across cloud environments. I began by deploying a virtual machine in Azure and implementing baseline security measures using Microsoft Defender for Cloud. This included enabling enhanced server protection features and configuring just-in-time (JIT) access to minimize exposure to external threats. I then progressed to integrating Microsoft Sentinel, where I onboarded data connectors, configured analytics rules, created playbooks for automated response, and tested custom alerts tied to suspicious activity. Each lab built upon the last, helping me understand how to layer proactive security, monitoring, and automation to better secure Azure workloads.

Lab 8 Azure Monitor

Exercise 1: Deploying an Azure virtual machine

Task 1: Deploying an Azure virtual machine

After signing into the Azure portal at <https://portal.azure.com> using the provided credentials—**LabUser-53202036@LODSPRODMCA.onmicrosoft.com** with the password **M+6a!jF2Jp3n**—I began the process of deploying a virtual machine as part of Exercise 1. This step was foundational for the monitoring and data collection configuration that would follow in later exercises.

I first opened the **Cloud Shell** by selecting the terminal icon in the top-right corner of the Azure portal interface. When prompted, I ensured that **PowerShell** was selected as the scripting environment. In the “Getting Started” window, I left the default configuration intact and chose to proceed without mounting any additional storage accounts, as persistent storage was not required for this particular lab session. From the drop-down menu, I selected the appropriate subscription associated with my lab environment—**lodssubscription**—and then clicked **Apply** to initialize the shell.

Once the Cloud Shell environment was ready, I started by creating a new resource group that would serve as the logical container for all resources used in Labs 8, 9, and 10. To do this, I ran the following PowerShell command directly in the shell:

```
New-AzResourceGroup -Name AZ500LAB131415 -Location 'EastUS'
```

This command provisioned a resource group named **AZ500LAB131415** in the **East US** region, which had been predesignated as the standard region for all lab resources. The command completed successfully, confirming that the resource group was in place and ready to host all

subsequent components, including the virtual machine, Log Analytics workspace, and storage account.

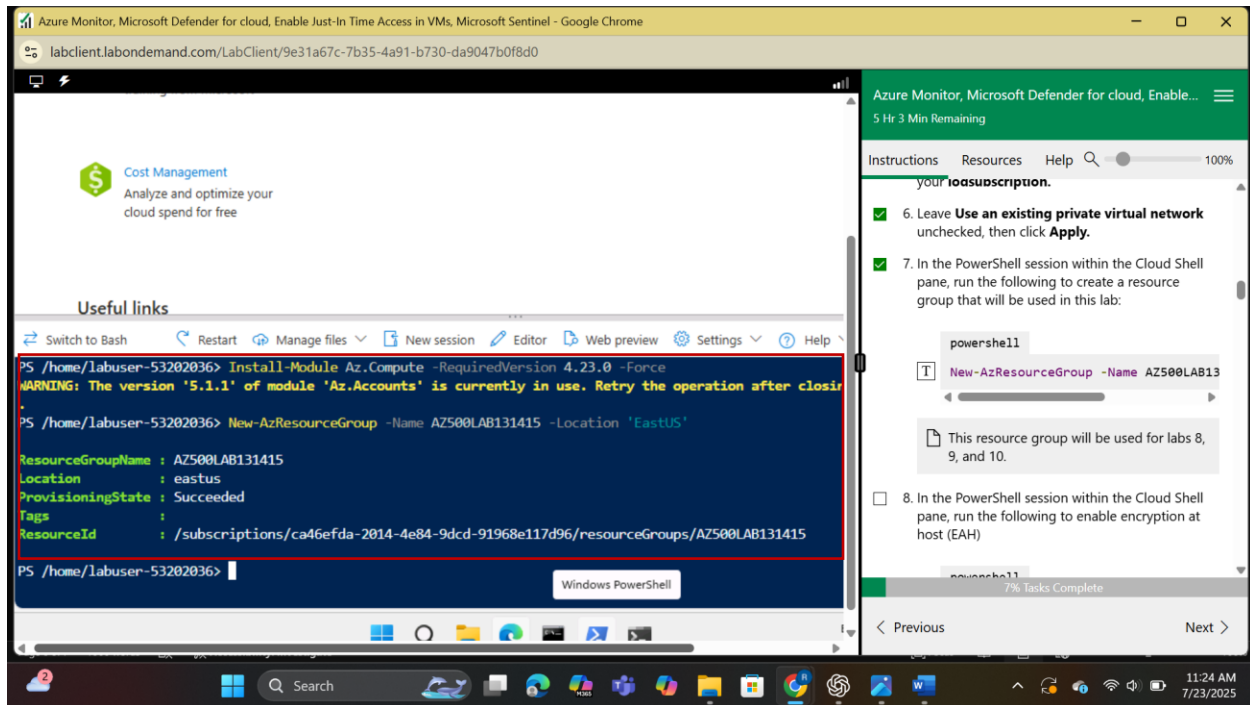


Fig 1.4 Successfully creating the lab resource

Next, I enabled a critical compute feature called **Encryption at Host (EAH)**. This feature ensures that data stored on the host machine running the virtual machine is encrypted at rest, thereby providing an additional layer of security beyond traditional disk-level encryption. To register this feature, I executed the following command:

```
Register-AzProviderFeature -FeatureName "EncryptionAtHost" -ProviderNamespace
Microsoft.Compute
```

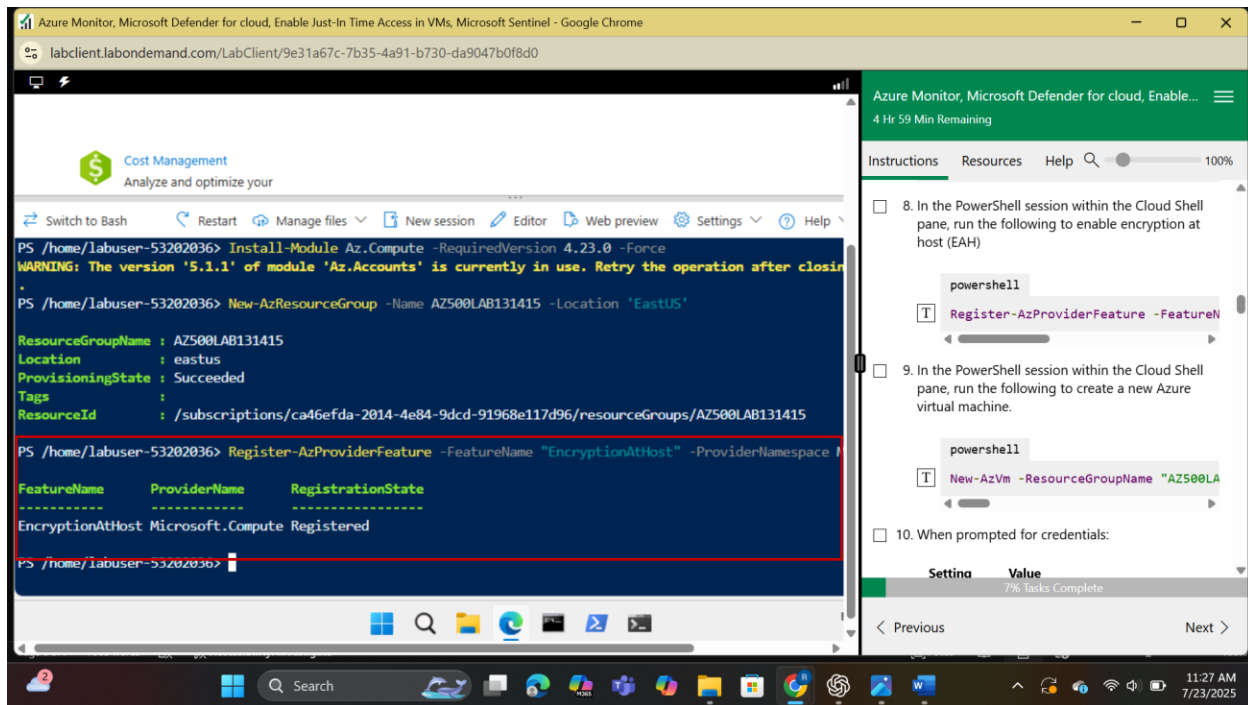



Fig 1.5 Enabling encryption at host (EAH)

Once the feature registration was initiated, I proceeded to deploy the virtual machine itself. I carefully constructed the VM deployment command, specifying all required parameters such as resource group, virtual network, subnet, public IP address, and open ports. The full command used to deploy the VM was:

```
New-AzVm -ResourceGroupName "AZ500LAB131415" -Name "myVM" -Location 'EastUS' -
VirtualNetworkName "myVnet" -SubnetName "mySubnet" -SecurityGroupName
"myNetworkSecurityGroup" -PublicIpAddressName "myPublicIpAddress" -PublicIpSku
Standard -OpenPorts 80,3389 -Size Standard_D2s_v3
```

Upon execution, I was prompted to enter credentials for the new virtual machine. I entered the username **localadmin** and used the custom password I had previously created in Lab 02, ensuring consistency with earlier configurations. The deployment process took a few minutes to complete, during which all the specified components—virtual network, subnet, network security group, public IP, and the virtual machine—were provisioned and linked together under the same resource group.

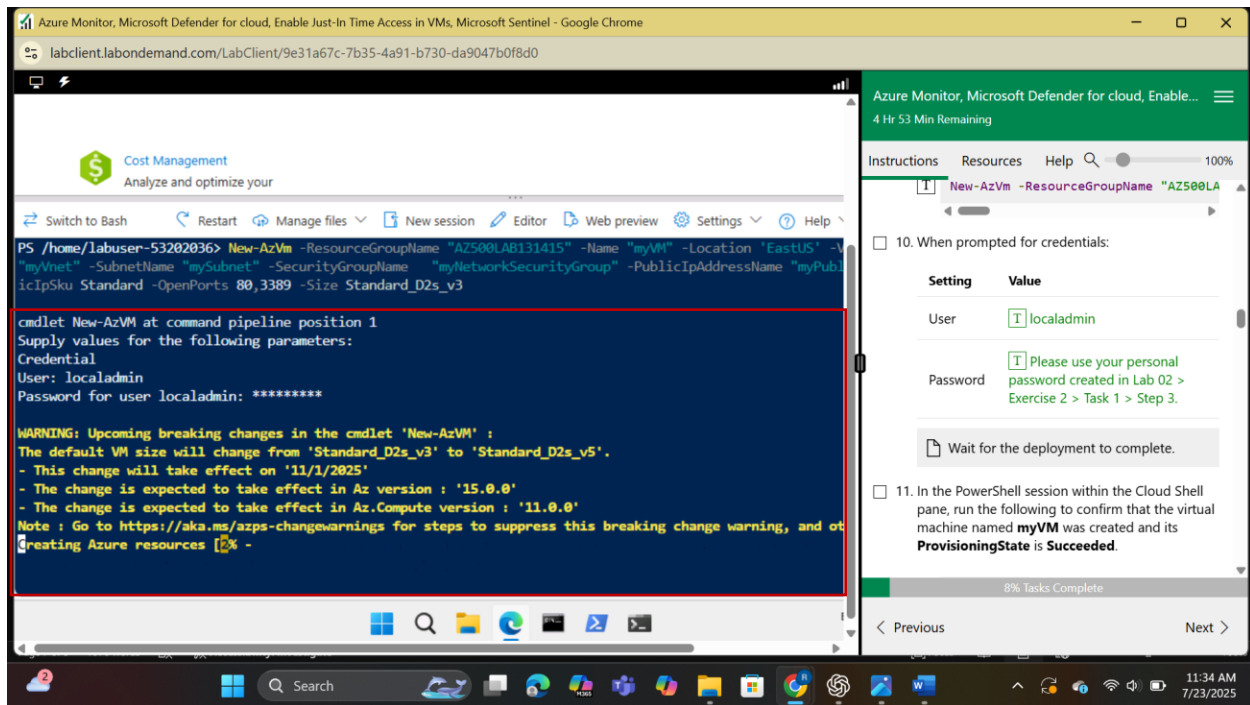


Fig 1.6 Creating a new Azure virtual machine.

To confirm that the VM deployment was successful, I ran the following verification command:

```
Get-AzVM -Name 'myVM' -ResourceGroupName 'AZ500LAB131415' | Format-Table
```

This command returned a formatted output showing the details of the virtual machine, including its provisioning state. I verified that the **ProvisioningState** was set to **Succeeded**, which confirmed that the VM was fully operational and ready for use in the next steps of the lab.

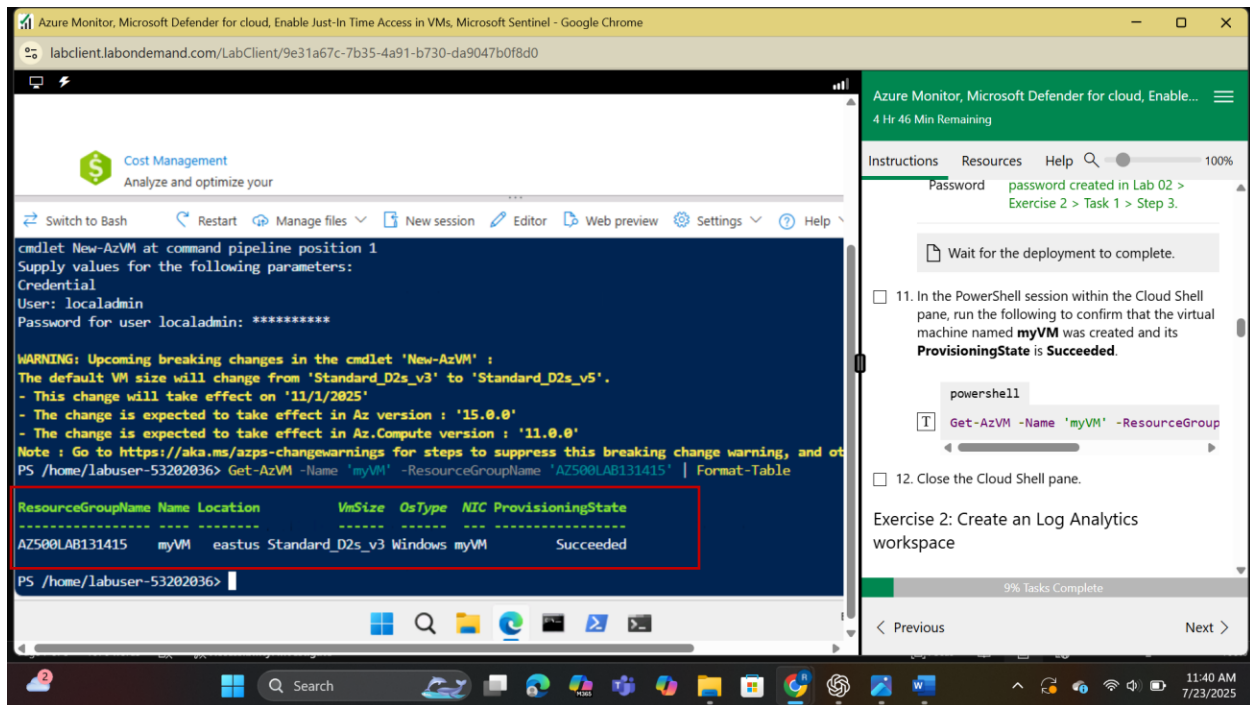


Fig 1.7 Confirming the deployment of the vm

After confirming the deployment, I closed the Cloud Shell pane, completing the first exercise with a clean and fully functional virtual machine environment ready for monitoring and data collection integration.

Exercise 2: Creating a Log Analytics Workspace

Task 1: Creating a Log Analytics Workspace

With the virtual machine successfully deployed and verified, I moved on to the next objective—creating a Log Analytics workspace that would serve as the central repository for monitoring data. This workspace is a key component in the Azure Monitor architecture, enabling the collection, analysis, and querying of log and performance data from connected Azure resources.

To begin, I navigated to the **Azure portal homepage** and used the global search bar at the top of the screen to search for “**Log Analytics workspaces.**” Once the service blade loaded, I selected the + **Create** button to initiate the workspace creation process.

On the **Basics** tab of the “Create Log Analytics workspace” blade, I specified the required configuration values for the workspace. I selected the appropriate **subscription** associated with my lab environment, and then selected the previously created resource group, **AZ500LAB131415**, to keep all resources logically grouped. For the workspace name, I entered **LAW53202036**, ensuring it met the naming standards. I also set the **Region** field to **East US**, aligning with the region used throughout the lab to avoid any cross-region dependency issues.

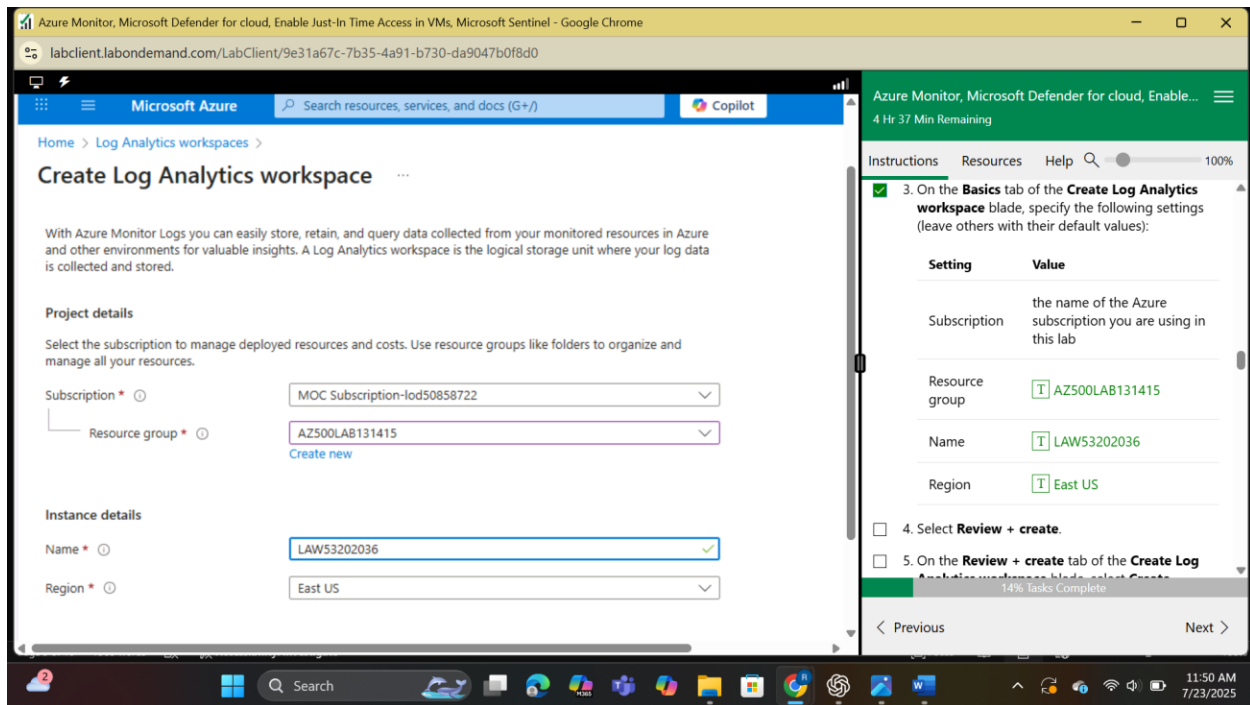


Fig 1.8 Filling in the relevant information on the Basics tab of the Create Log Analytics workspace blade

After entering the values, I clicked **Review + create**, and once the validation passed, I proceeded by selecting the **Create** button. The deployment process was initiated and completed within a short time, after which I was able to confirm that the workspace **LAW53202036** appeared in the list of available Log Analytics workspaces under the correct subscription and resource group.

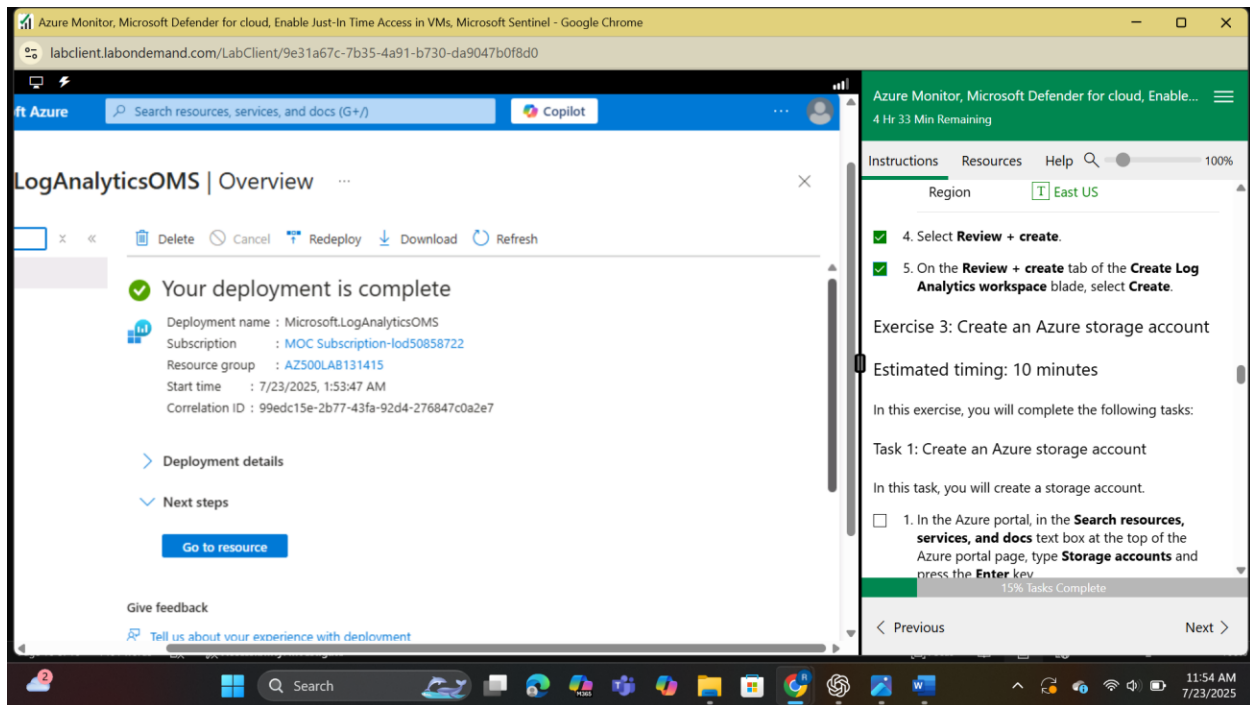


Fig 1.9 Successful deployment of the log analytic work space

With the workspace now in place, I had successfully established the foundational element required for log ingestion and data collection in Azure Monitor. This workspace would later be linked to data collection rules and virtual machines to gather metrics and logs essential for visibility, threat detection, and performance tuning.

Exercise 3: Creating an Azure Storage Account

Task 1: Creating an Azure Storage Account

Following the successful creation of the Log Analytics workspace, I proceeded to configure an Azure Storage account, which would serve as a supplementary data storage component for logs and diagnostic data. Storage accounts in Azure provide scalable and secure cloud storage for a wide range of data types, and in this context, they are essential for persisting certain types of collected telemetry.

From the Azure portal, I once again used the search bar at the top of the interface and typed “**Storage accounts**”. When the blade loaded, I clicked the + **Create** button to initiate the setup of a new storage account.

On the **Basics** tab of the “Create storage account” blade, I configured the required settings. I ensured that the correct **subscription** was selected, and then chose the **AZ500LAB131415** resource group to maintain consistency across all deployed resources. For the **Storage account name**, I entered a globally unique string composed of lowercase letters and numbers, as required

by Azure naming conventions. I ensured that the name was between 3 and 24 characters in length and had not been used previously.

For the region, I once again selected **East US** to remain consistent with the rest of the lab setup. Under **Performance**, I retained the default setting of **Standard**, which provides general-purpose v2 storage optimized for cost-efficiency and latency. Under the **Redundancy** option, I selected **Locally Redundant Storage (LRS)**, which replicates data within a single datacenter to protect against drive failures while minimizing cost.

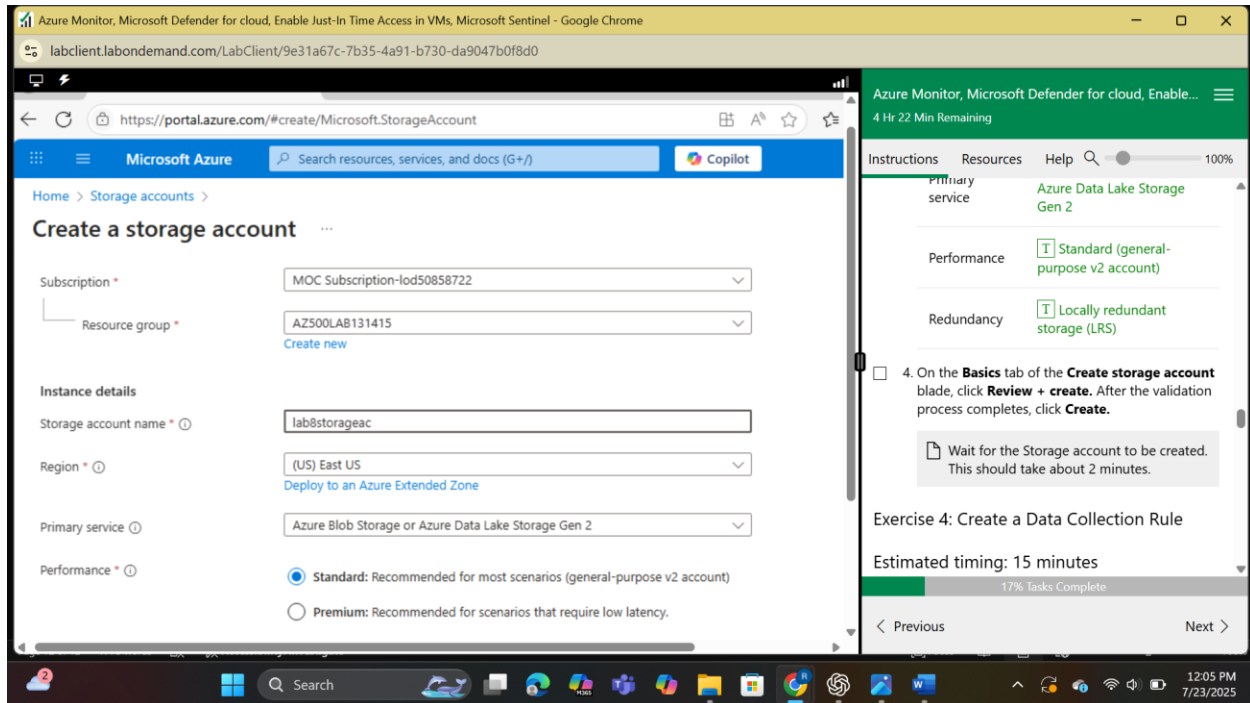


Fig 2.0 Filling in the required details on azure storage account

After configuring these basic parameters, I clicked **Review + create**, and after a successful validation, selected **Create** to begin the deployment. The provisioning process took approximately two minutes, after which the storage account was visible under the list of accounts in the portal, confirming a successful deployment.

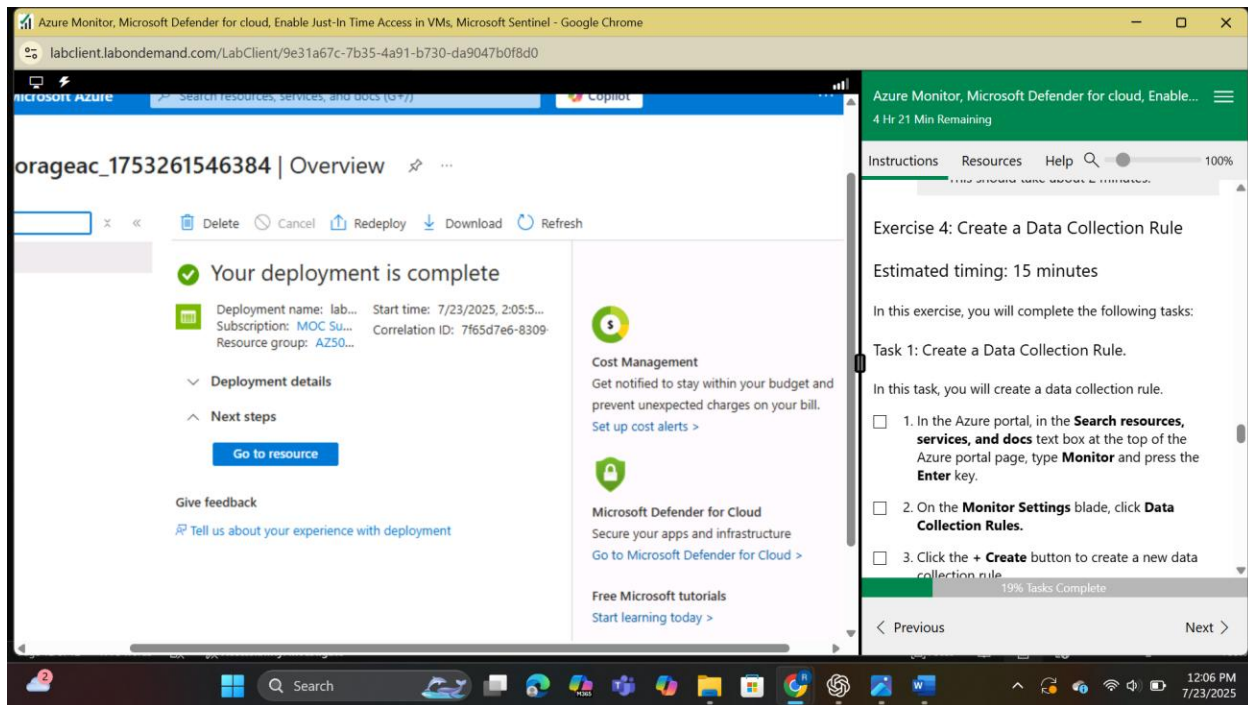


Fig 2.1 Complete deployment of the account

This storage account would later be available for use in diagnostic settings, backup configurations, or additional telemetry collection tasks as needed in subsequent lab exercises. With this step complete, the environment was now equipped with a dedicated workspace and persistent storage layer, both critical components of a robust monitoring and logging solution.

Exercise 4: Creating a Data Collection Rule

Task 1: Creating a Data Collection Rule

With both the Log Analytics workspace and the storage account configured, I moved on to setting up a Data Collection Rule (DCR), which is essential for managing what telemetry is collected from Azure resources and where that data is sent. In this case, the goal was to gather performance counters from virtual machines and route them to the previously created Log Analytics workspace for centralized analysis through Azure Monitor.

To begin, I returned to the Azure portal homepage and used the search bar to look for “**Monitor.**” Once in the Azure Monitor blade, I scrolled through the settings pane and selected **Data Collection Rules**. From there, I clicked on the + **Create** button to begin configuring a new DCR.

On the **Basics** tab of the “Create Data Collection Rule” wizard, I provided the following configuration details: for the **Rule Name**, I entered **DCR1**. I confirmed the correct **subscription** and chose the existing **AZ500LAB131415** resource group to ensure that the rule would be

created in the same logical boundary as the other lab resources. I set the **Region** to **East US**, matching all previously created components, and selected **Windows** as the **Platform Type**, since the target virtual machine is running the Windows operating system. I left the **Data Collection Endpoint** field blank as instructed.

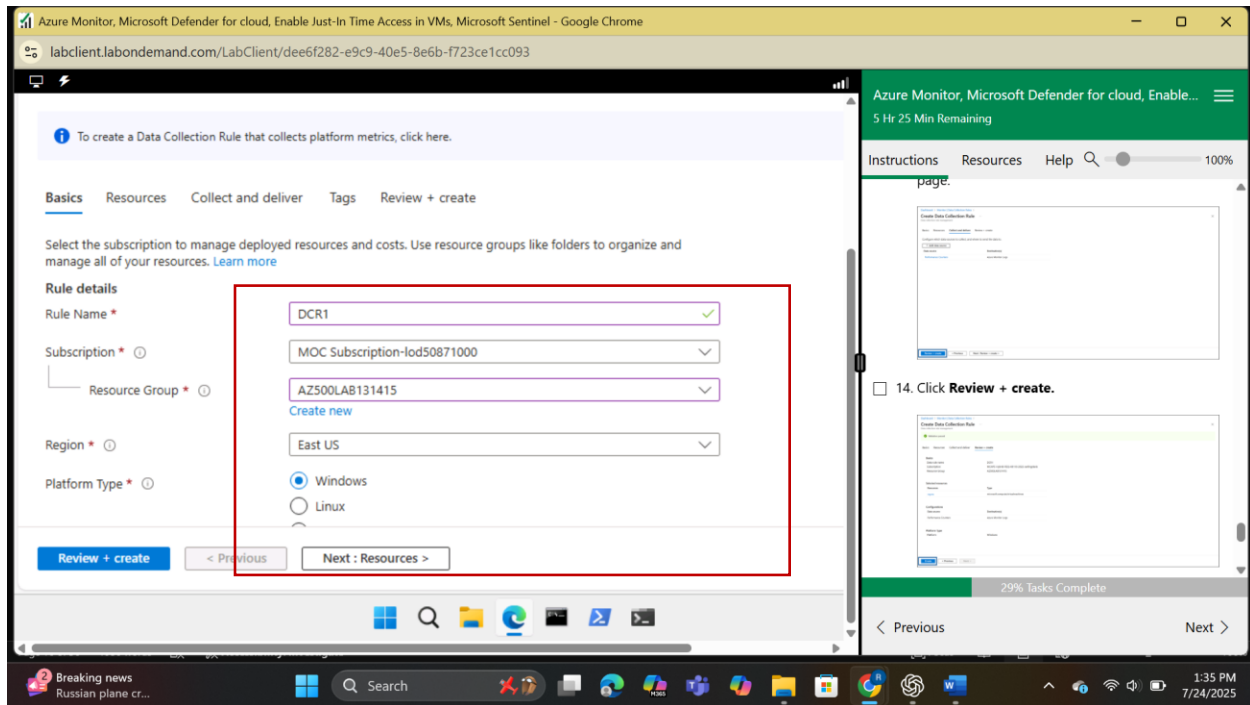


Fig 2.2 Information on the basics blade

With the basic information in place, I clicked **Next: Resources >** to move to the next section. On the **Resources** tab, I selected **+ Add resources**, and in the scope template, I checked the **Subscription** checkbox. After verifying that the subscription scope was correctly set, I clicked **Apply** to proceed.

Next, on the **Collect and deliver** page, I clicked **+ Add data source**, then selected **Performance Counters** from the **Data source type** drop-down menu. I was presented with a list of standard performance counters used in monitoring system health, and I kept the default sample rate of **60 seconds** for each counter type. The included performance counters were **CPU**, **Memory**, **Disk**, and **Network**, all of which are essential metrics for establishing a baseline of virtual machine performance and detecting abnormal behavior.

Once the data sources were configured, I clicked **Next: Destination >**. On this screen, I clicked **+ Add destination**, then selected **Azure Monitor Logs** as the destination type. Under the **Account or namespace** drop-down, I selected the **LAW53202036** Log Analytics workspace created earlier. This binding ensured that all performance data collected under this rule would be routed directly into the Log Analytics environment for querying, dashboarding, and alerting.

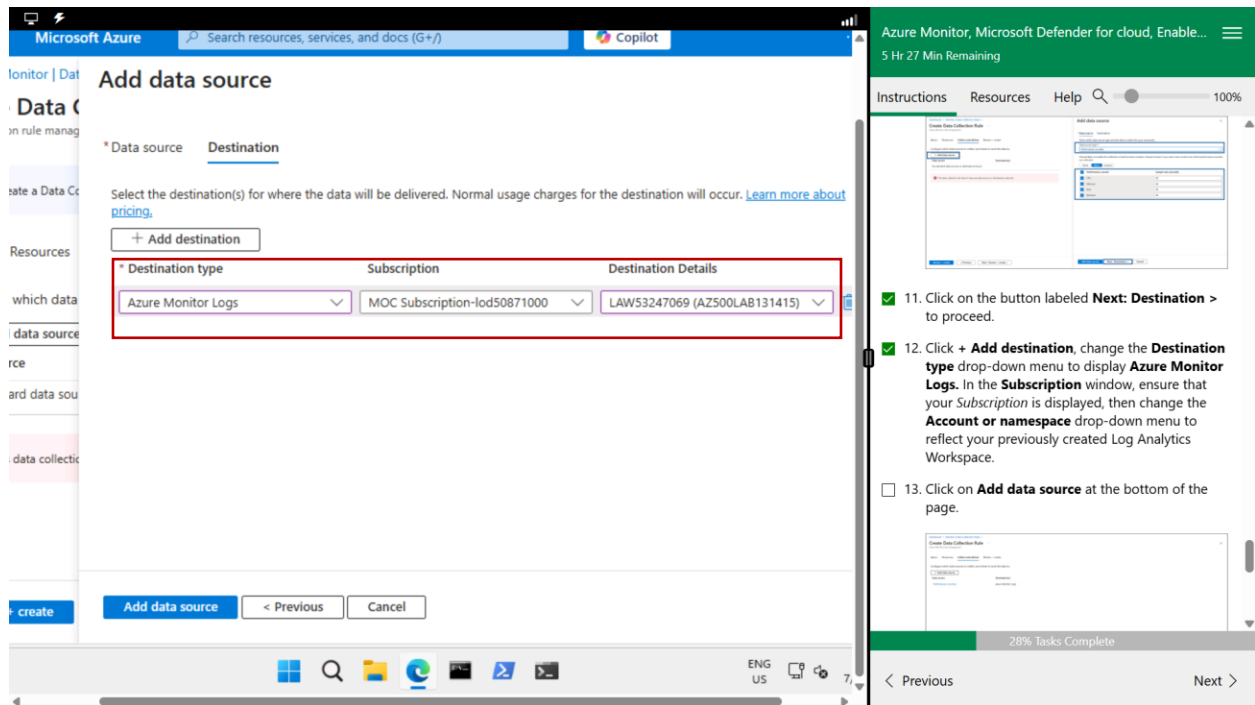


Fig 2.3 Information on the resource blade

I reviewed the configuration on the final tab and then clicked **Create** to deploy the Data Collection Rule. The deployment completed successfully after a brief provisioning period, confirming that the rule was now active and ready to begin collecting telemetry from supported virtual machines.

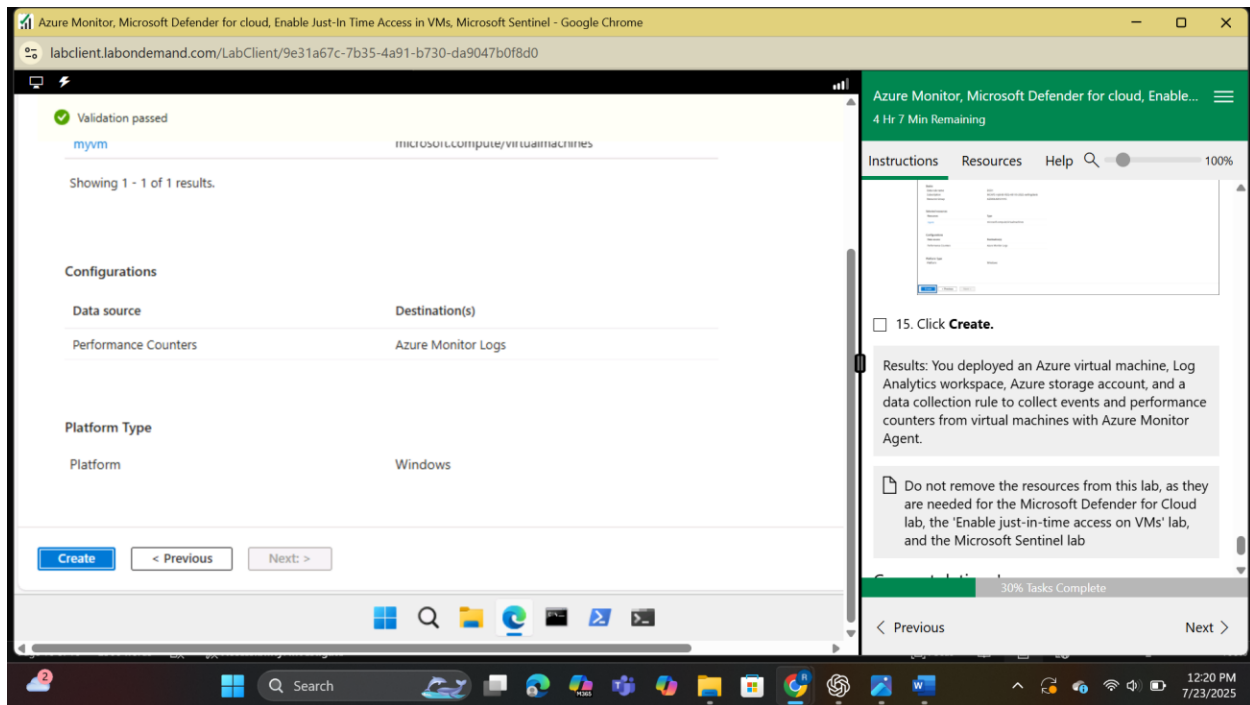


Fig 2.4 Reviewing the configuration and clicking create

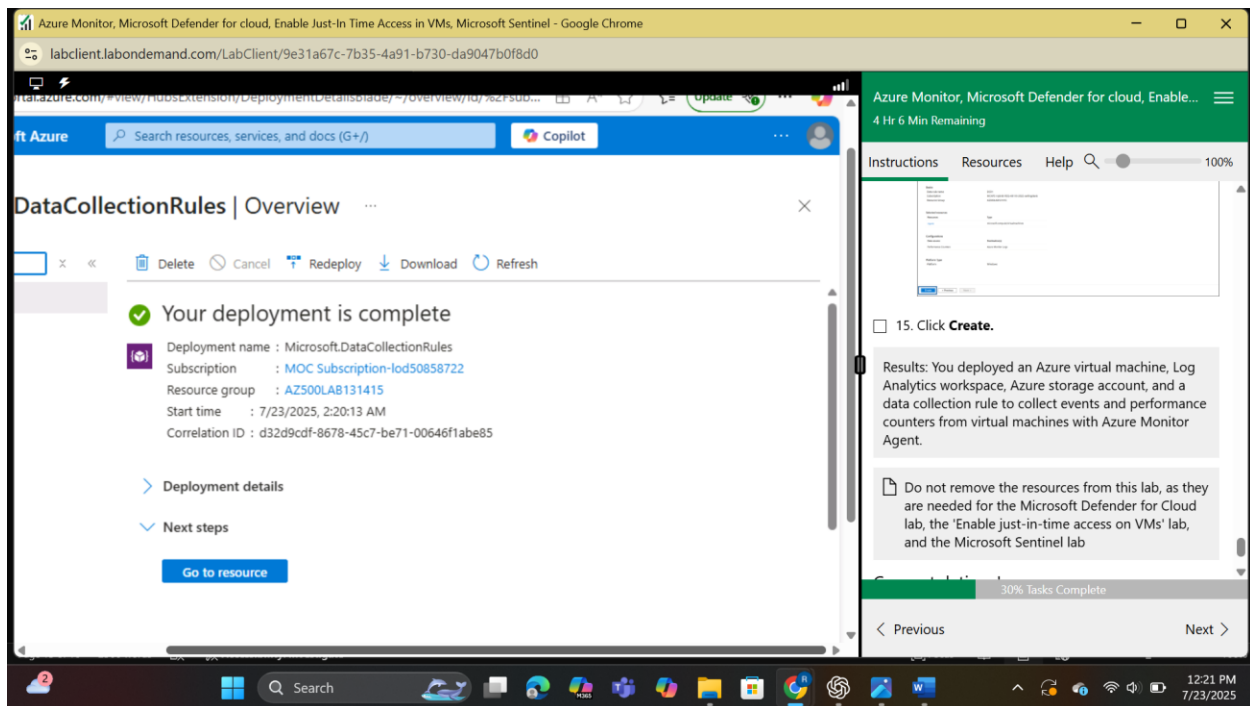


Fig 2.5 Successful deployment of the Microsoft data collection rules

With the DCR now in place, I had established an automated pipeline for collecting, routing, and analyzing performance data—enabling real-time observability and supporting advanced analytics through Azure Monitor. This completed the last configuration task of the lab and fully set up the

monitoring infrastructure needed for secure and reliable VM operations in the Azure environment.

Lab 09: Configuring Microsoft Defender for Cloud Enhanced Security Features for Servers

Exercise 1: Configuring Microsoft Defender for Cloud Enhanced Security Features for Servers

To begin the process of enabling enhanced protection for virtual machines, I signed into the Azure portal and accessed **Microsoft Defender for Cloud** by typing the service name into the global search bar at the top of the interface. Once the blade loaded, I navigated to the **Management** section and selected **Environment settings**, which is where security configurations for Azure subscriptions are managed.

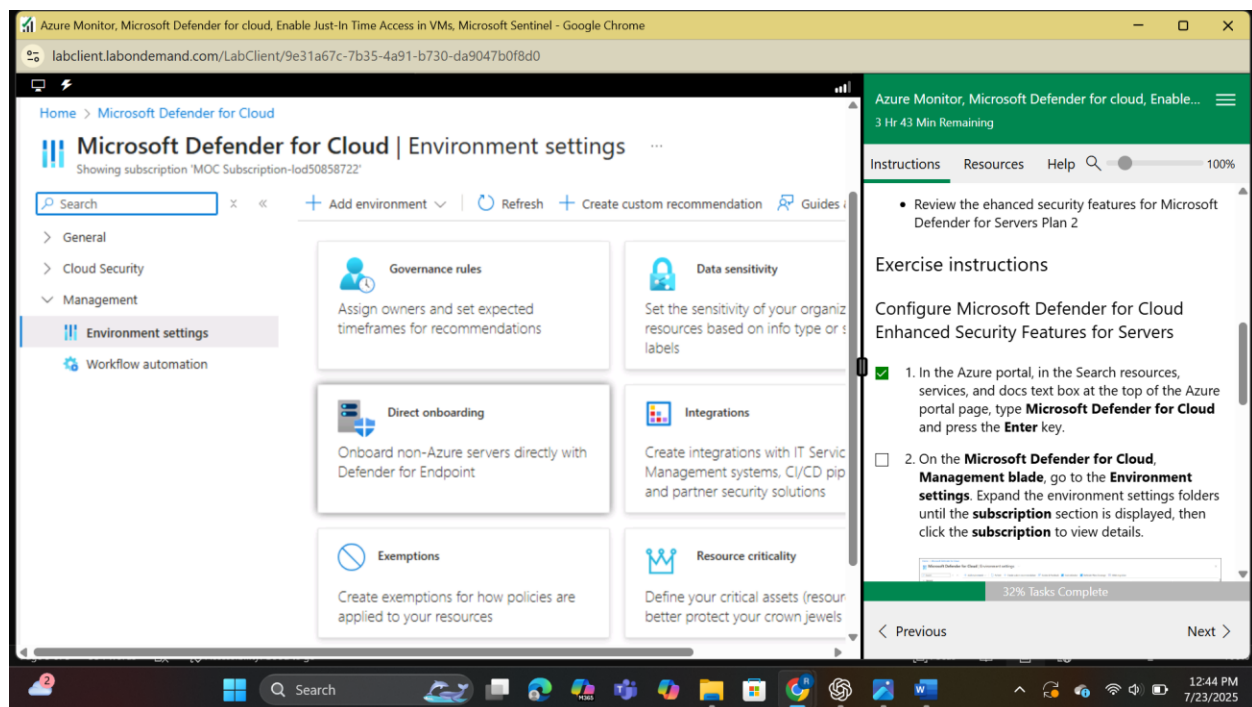


Fig 2.6 The environment setting blade

In the Environment settings pane, I expanded the folder structure until I located and clicked on the subscription used for the lab. This brought up a detailed settings view specifically for that subscription. From there, I located the **Defender plans** section under **Settings**, and expanded the **Cloud Workload Protection (CWP)** category. This section included all available workload-specific Defender plans.

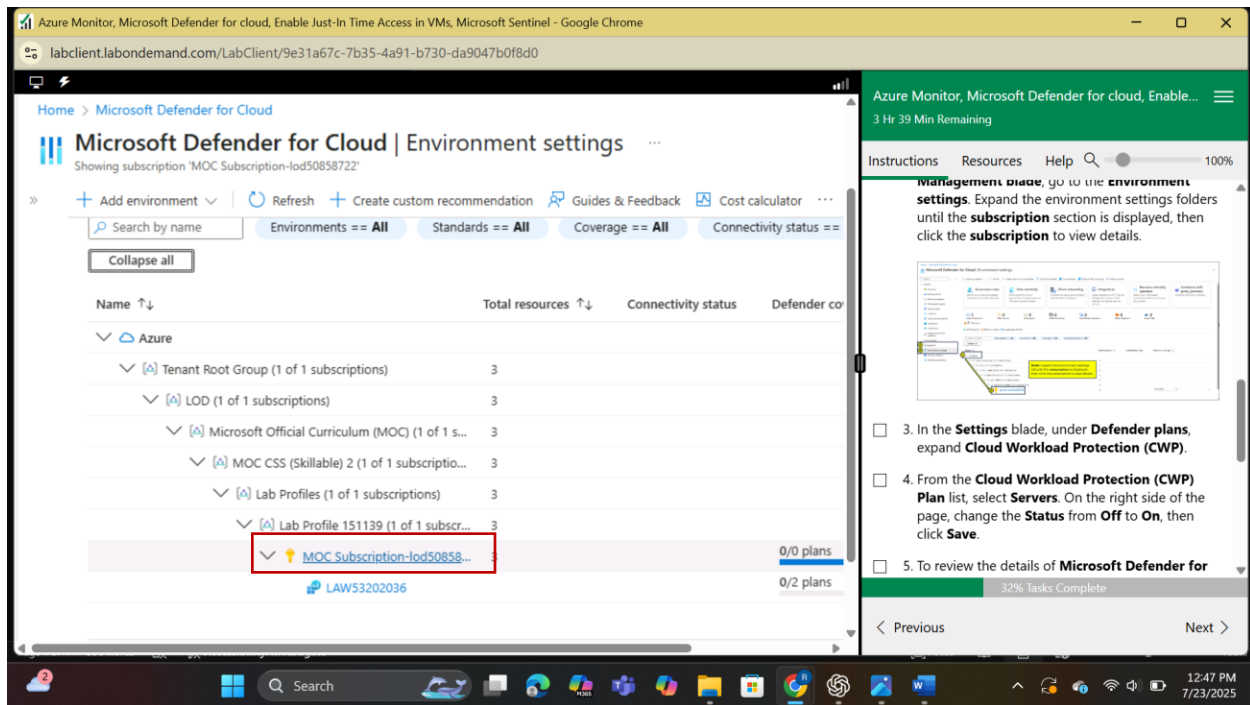


Fig 2.7 Locating the subscription used

Within the CWP list, I selected **Servers**, which opened a configuration panel on the right side of the screen. The current status of the plan was set to **Off**, so I changed it to **On** to activate protection. After toggling the switch, I clicked **Save** to confirm the change. This action enabled **Microsoft Defender for Servers Plan 2**, which provides advanced security features such as threat detection, adaptive application controls, and just-in-time VM access.

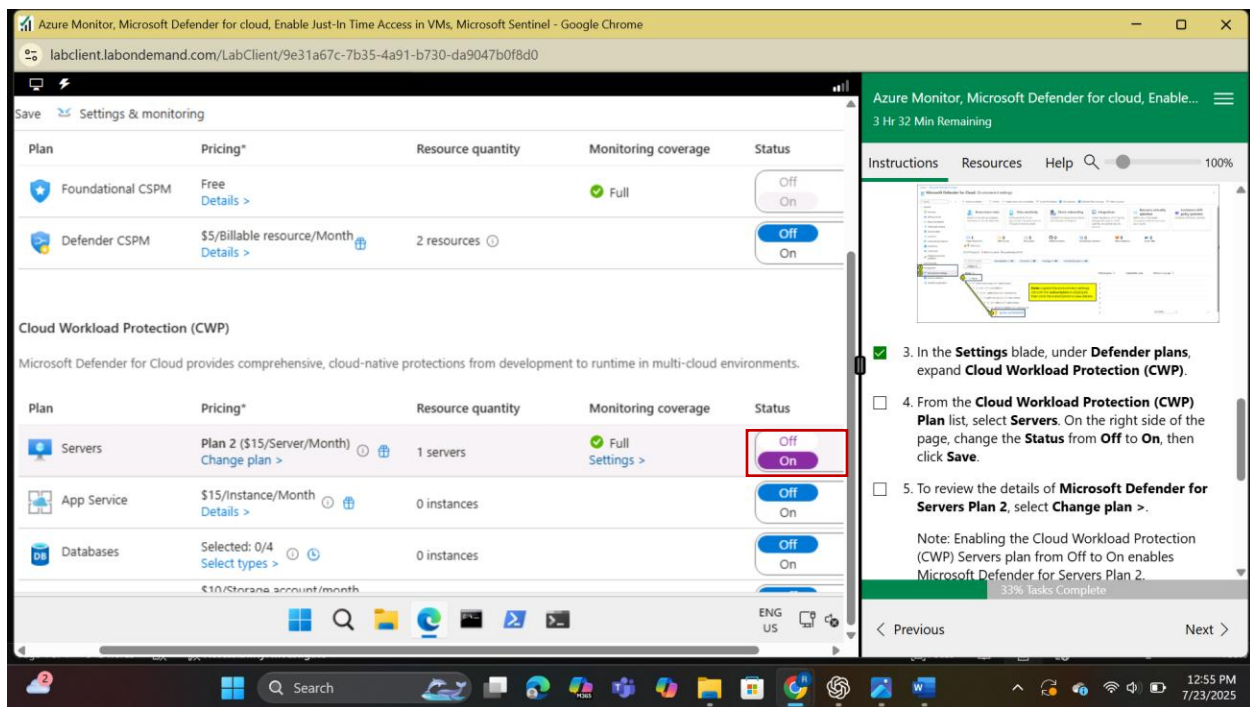


Fig 2.8 Turning the serve status to on

To further review what was included in Plan 2, I clicked on the **Change plan** option. This provided a detailed breakdown of the capabilities enabled by the plan, including integration with Microsoft Defender for Endpoint, file integrity monitoring, and automatic provisioning of the Azure Monitor agent.

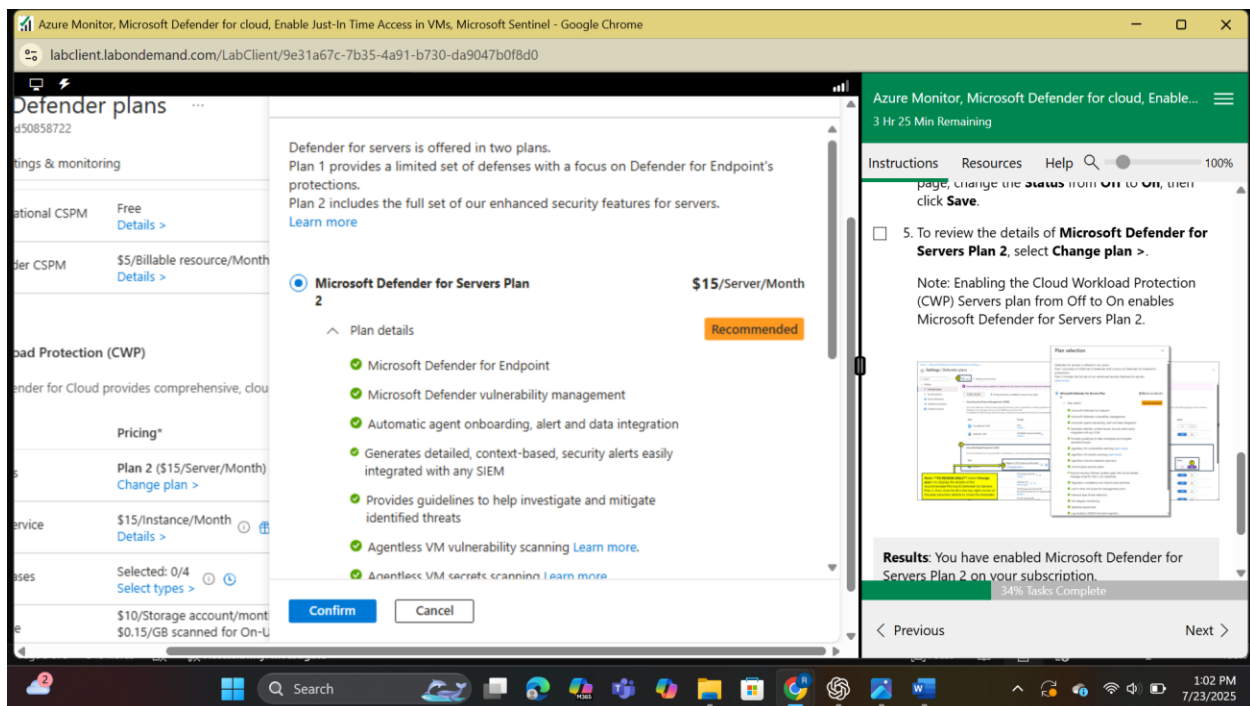


Fig 2.9 Reviewing the details of Microsoft Defender for Servers Plan 2

With this configuration in place, the subscription was now protected by Microsoft Defender for Servers Plan 2, ensuring that all Azure virtual machines under its scope benefit from enhanced threat protection, real-time security alerts, and improved visibility into vulnerabilities and misconfigurations. This completed the lab, successfully enabling a critical layer of defense for hybrid and cloud-based server workloads.

Lab 10: Enabling Just-In-Time Access on VMs

Exercise 1: Enabling JIT on your VMs from Azure Virtual Machines

To begin configuring Just-in-Time (JIT) access for my virtual machine, I navigated to the **Virtual machines** blade by entering “virtual machines” into the global search bar and selecting the appropriate result. From the list of available VMs, I selected **myVM**, which had been previously deployed in Lab 08 and was now ready for security enhancements.

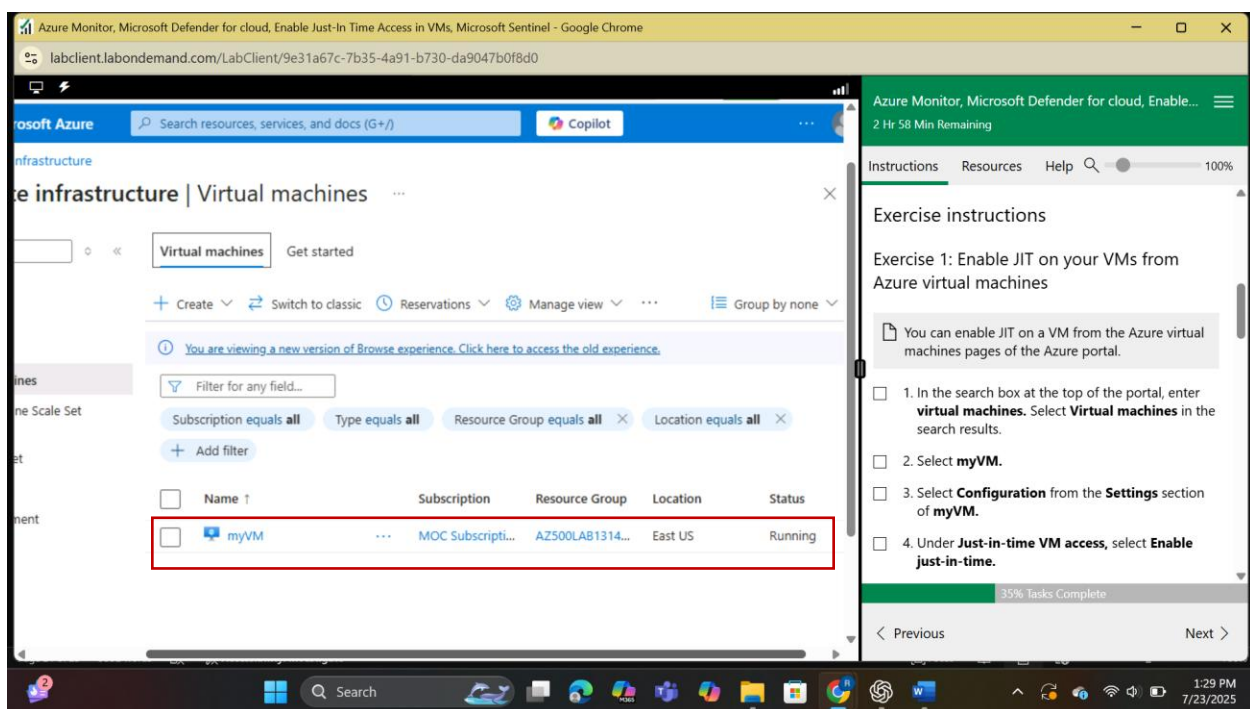


Fig 3.0 Previously deployed, myVM

Once inside the myVM resource page, I scrolled down to the **Settings** section and clicked on **Configuration**. Under the **Just-in-time VM access** option, I located and selected the **Enable just-in-time** link. This action redirected me to the **Microsoft Defender for Cloud** interface,

where advanced JIT configurations could be applied. The integration between Azure VM settings and Defender for Cloud made it seamless to enable this feature directly from the VM context.

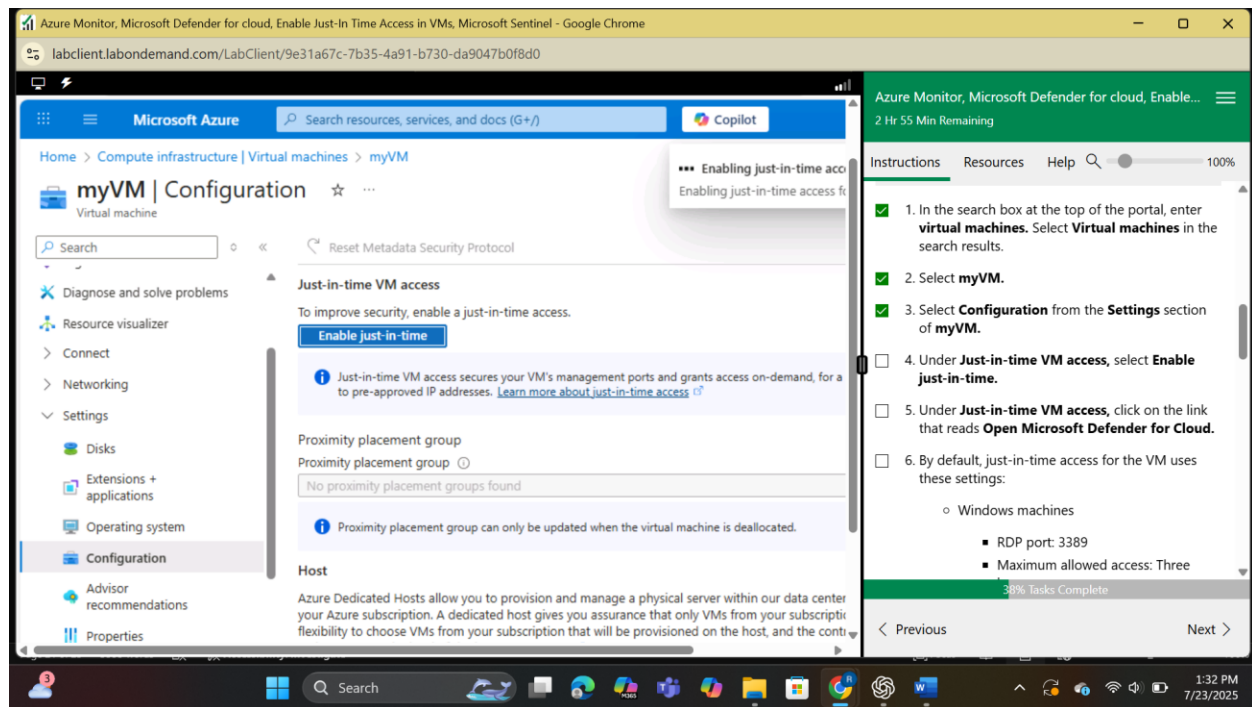


Fig 3.1 Enabling JIT

By default, JIT was configured to protect the most commonly targeted management ports. For this Windows-based machine, that included **TCP port 3389** for Remote Desktop Protocol (RDP). The default rules allowed JIT requests for a maximum of **three hours** and from **any source IP address**, which is useful for initial setup but would typically be tightened in a production scenario.

To view these settings, I went to the **Configured** tab in the JIT configuration panel. From there, I right-clicked on the entry for **myVM** and selected **Edit**. This brought up the configuration interface where I could modify existing port rules or add new ones. Although the defaults were sufficient for this lab, this step demonstrated how additional ports—such as custom application or management ports—could be added and controlled with the same JIT principles. Once reviewed, I selected **Save** to apply the JIT policy to the VM.

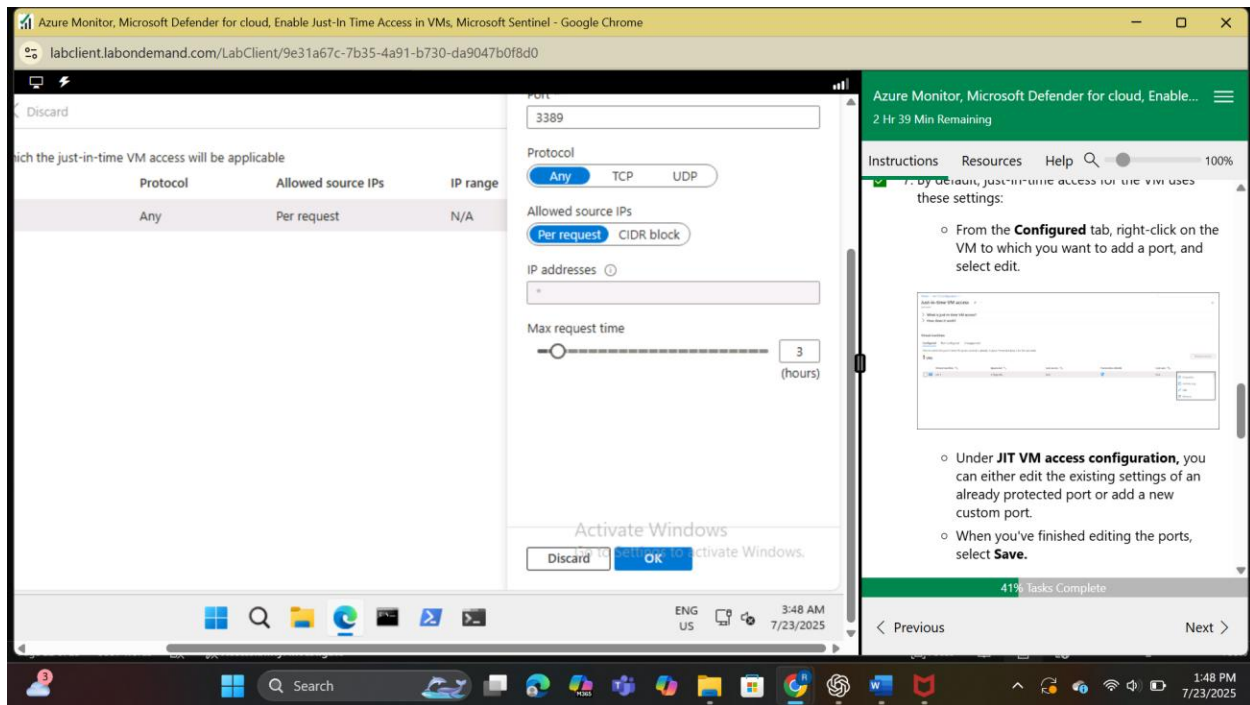


Fig 3.2 Viewing the VM's configurations

At this point, JIT VM access was successfully enabled for myVM. The configuration ensured that inbound traffic to critical ports like RDP would only be allowed when explicitly requested, reducing the attack surface and mitigating risks such as brute-force attacks. This laid the groundwork for secure, time-bound administrative access to the VM, which I would test in the next exercise.

Exercise 2: Requesting Access to a JIT-Enabled VM from the Azure Virtual Machine's Connect Page

After successfully enabling Just-in-Time (JIT) VM access on **myVM**, I proceeded to validate the configuration by requesting access through the Azure portal. This step ensures that even though JIT restricts open port access by default, administrators can still securely connect to VMs when necessary by issuing approved access requests.

To begin, I returned to the **Virtual machines** blade in the Azure portal and selected **myVM** from the list. Once inside the VM overview page, I clicked on **Connect** in the top menu — the standard way to initiate a Remote Desktop session for Windows VMs. As expected, Azure immediately detected that JIT was enabled for this VM.

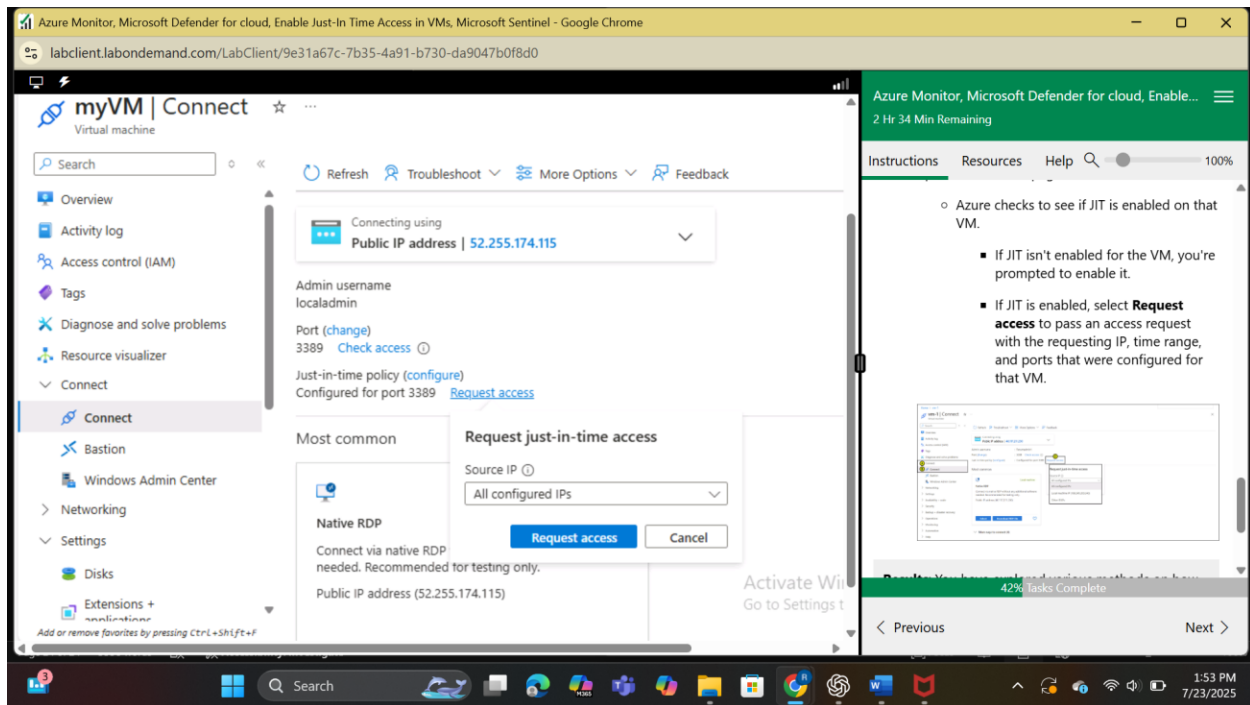


Fig 3.3 Jit enabled on the vm

Instead of allowing a direct connection, the portal prompted me to **Request access**, which launched a configuration panel. Here, I confirmed the parameters of my access request. The portal pre-filled the **RDP (3389)** port, and I left the access duration at the default of **3 hours**. I also left the **source IP address setting** as "Any" to keep the configuration consistent with the lab's defaults.

Once I reviewed these settings, I clicked **Request access**. Azure Defender for Cloud processed the request, and within seconds, the firewall rules were temporarily opened to allow the connection from my current IP address. This confirmed that JIT access was not only enabled but functional, providing secure, on-demand administrative access while minimizing exposure.

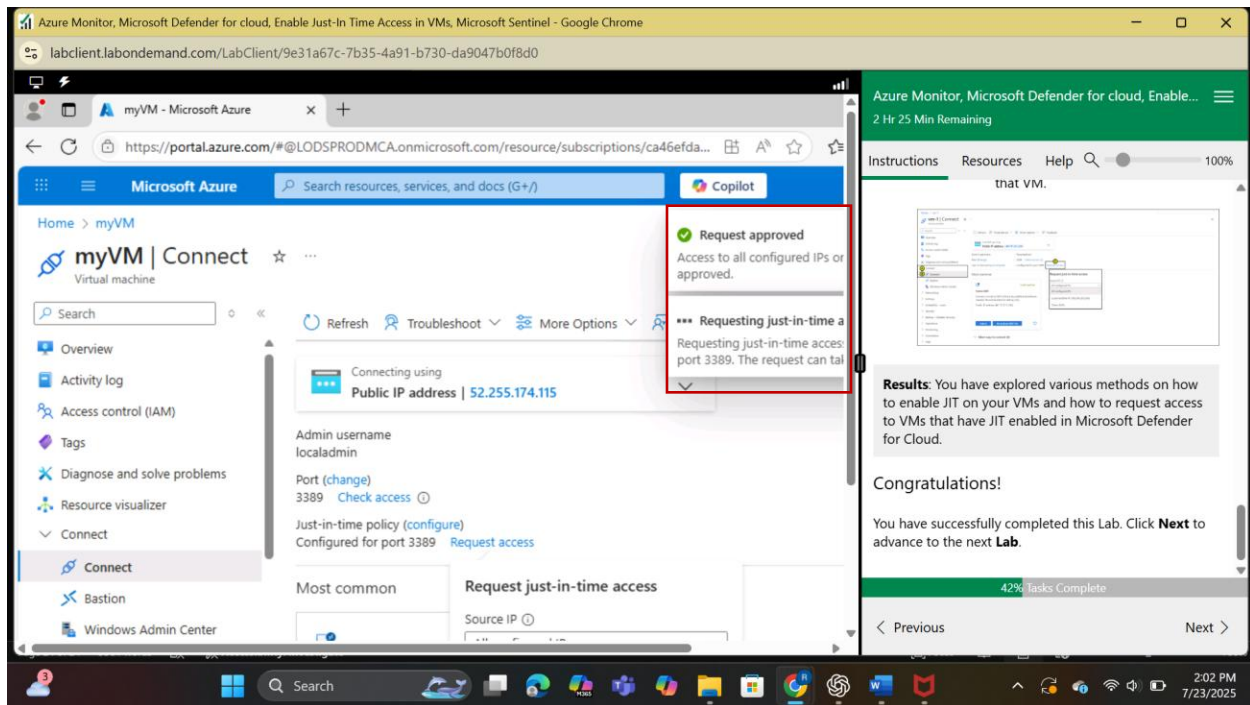


Fig 3.4 Request and approval of access to all configured ip

With this test, I verified that the JIT configuration worked as intended: by default, critical ports remain closed, and temporary access can be granted in a controlled and time-bound manner. This effectively reduces the window of opportunity for potential attacks while still supporting operational needs.

By the end of this exercise, I had successfully demonstrated the process of both enabling and utilizing JIT VM access — a powerful security measure that aligns with best practices for privileged access management in Azure.

Lab 11: Microsoft Sentinel

In this lab, I implemented Microsoft Sentinel to create a proof of concept for threat detection and automated response. The goal was to start collecting data from Azure Activity and Microsoft Defender for Cloud, add both built-in and custom alerts, and review how playbooks can automate responses to incidents. All resources were created in the East US region.

Exercise 1: Onboarding Microsoft Sentinel

To begin, in the Azure portal, I searched for "Microsoft Sentinel" in the top search bar and selected it from the services list. On the Microsoft Sentinel blade, I clicked "+ Create" to add Sentinel to a workspace. I selected the Log Analytics workspace that I had created during the Azure Monitor lab and then clicked Add. I ensured that the workspace met Sentinel's

requirements, such as not being created by Microsoft Defender for Cloud, which is a limitation for compatibility.

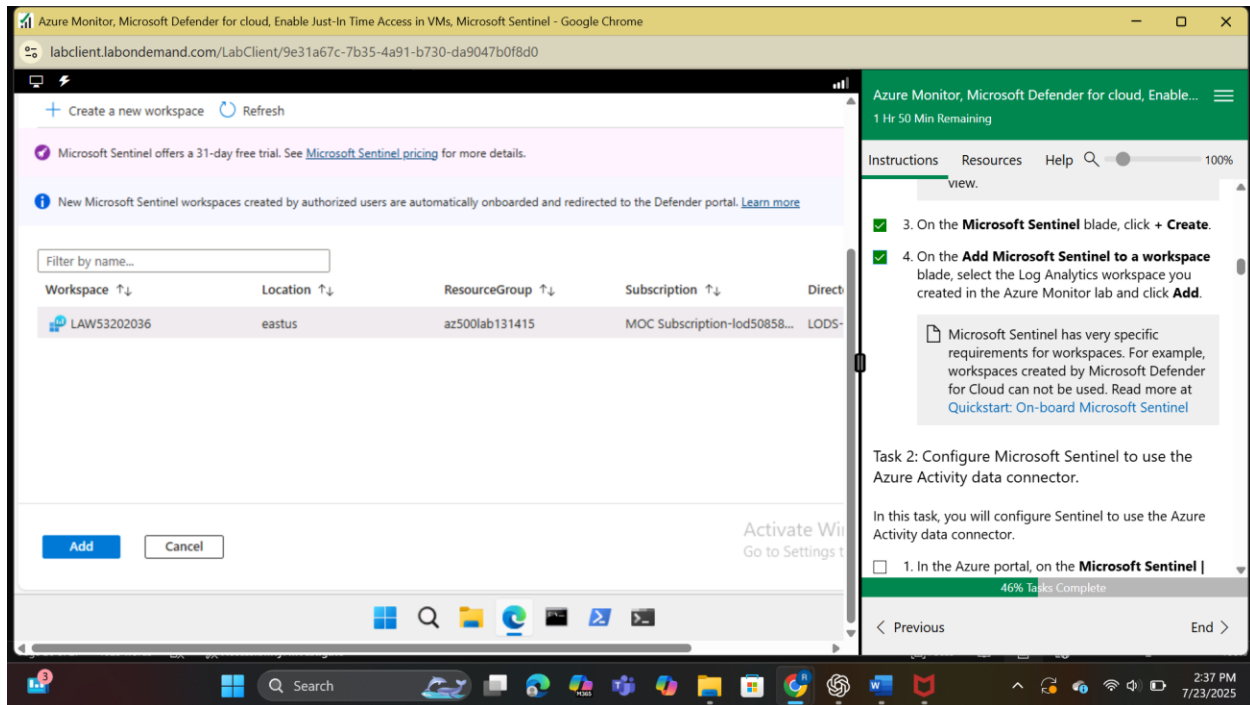


Fig 3.5 Adding Log Analytics workspace created earlier

Exercise 2: Connecting Azure Activity to Sentinel

After onboarding, I configured Microsoft Sentinel to use the Azure Activity data connector. From the Sentinel Overview blade, I clicked Content hub under the Content management section. In the Content hub, I searched for "Azure Activity," reviewed the description, and clicked Install.

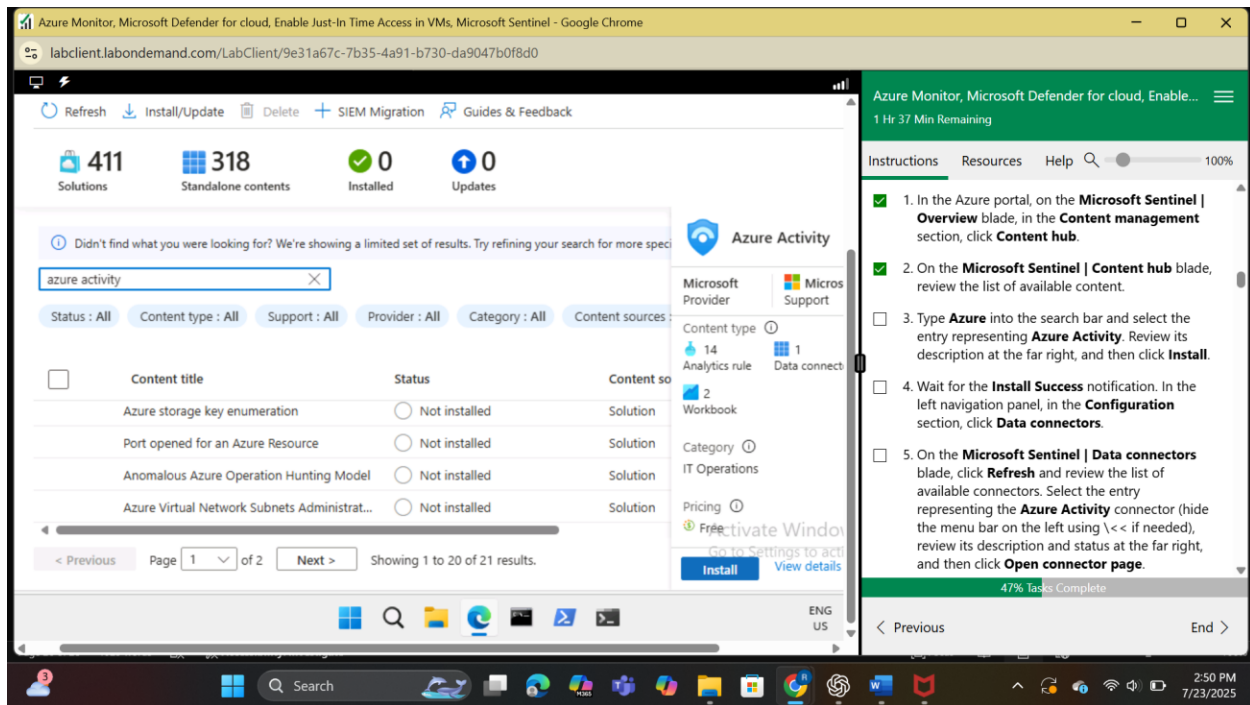


Fig 3.6 Installing azure activity

Once installed, I navigated to the Data connectors section and refreshed the page to see the updated list. I selected the Azure Activity connector and clicked Open connector page. On the Azure Activity connector page, under the Instructions tab, I skipped step 1 since the Disconnect All button was greyed out, indicating no legacy connectors were used. In step 2, I clicked "Launch the Azure Policy Assignment wizard" and on the Basics tab, I set the scope by selecting my subscription without choosing a resource group. I clicked Next twice to reach the Parameters tab and then selected my Log Analytics workspace.

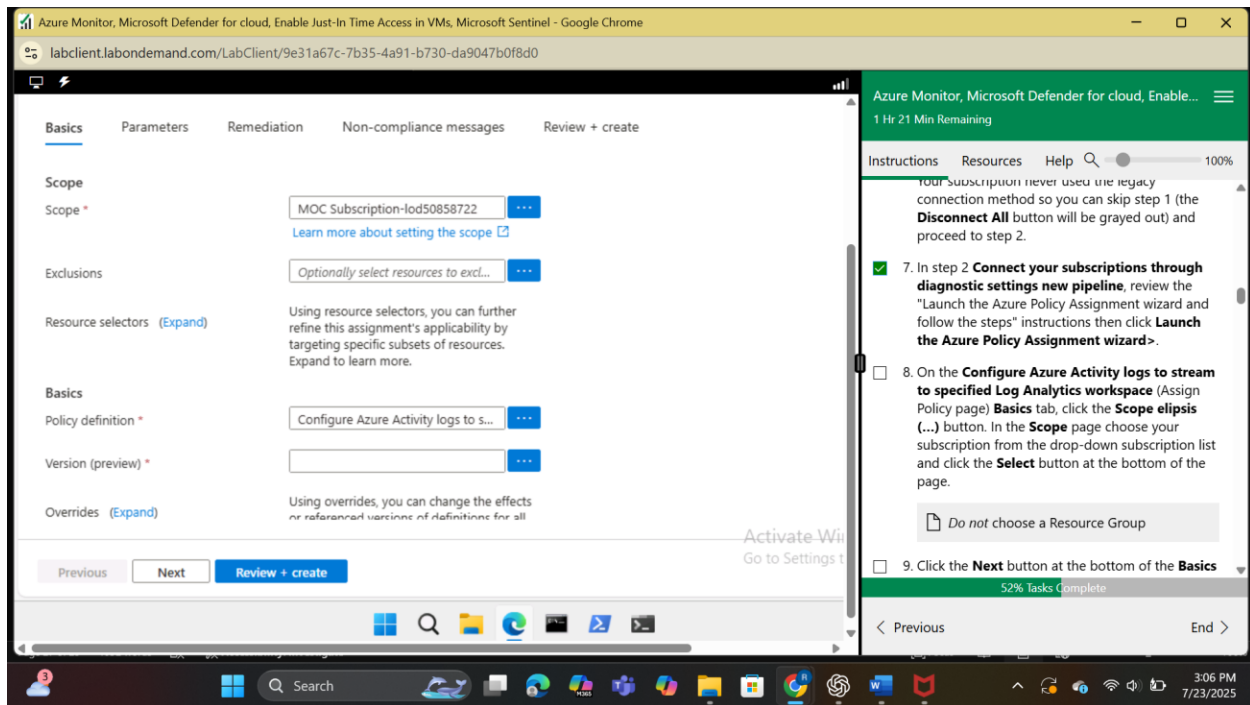


Fig 3.7 Filling in the required information on the Azure Policy Assignment wizard

On the Remediation tab, I checked the box to create a remediation task and set the system-assigned identity location to East US. I skipped the optional non-compliance message and clicked Review + Create, then Create. I verified successful creation of the policy assignment, role assignments, and remediation task by checking the Azure notifications bell icon. I waited for data to be received, which was confirmed by the graph appearing on the Azure Activity connector pane after refreshing.

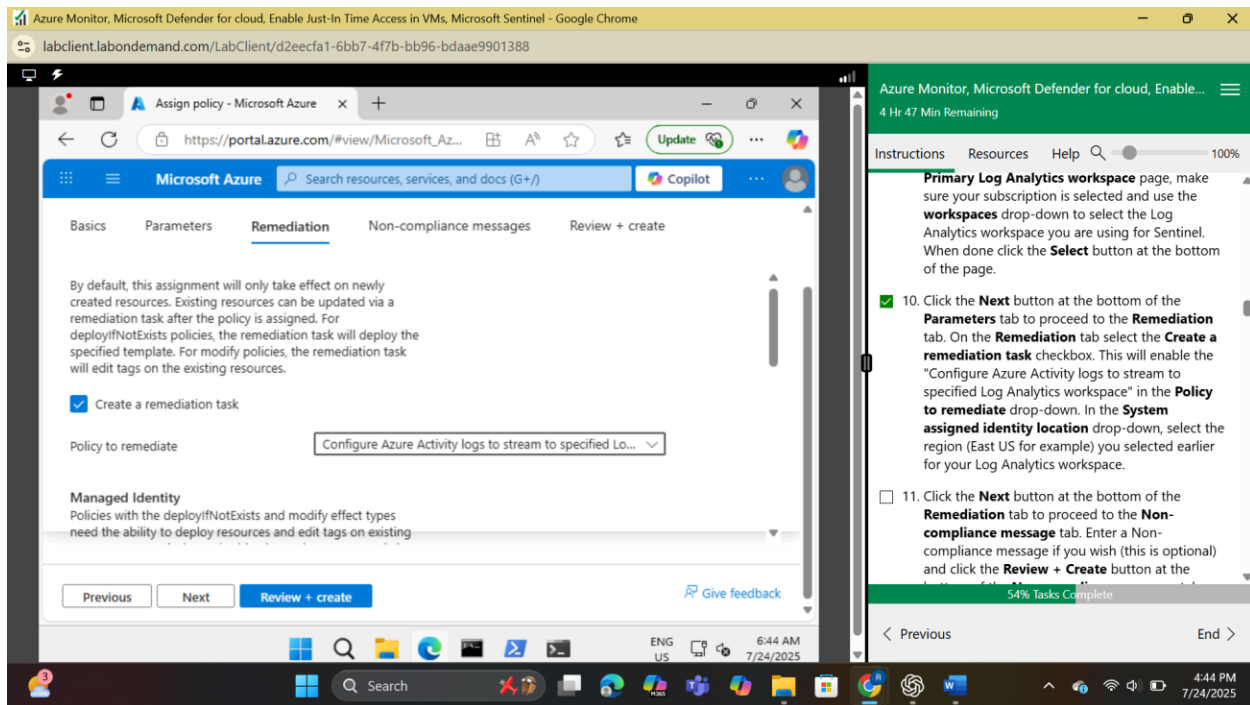


Fig 3.8 Filling in required information on remediation pane

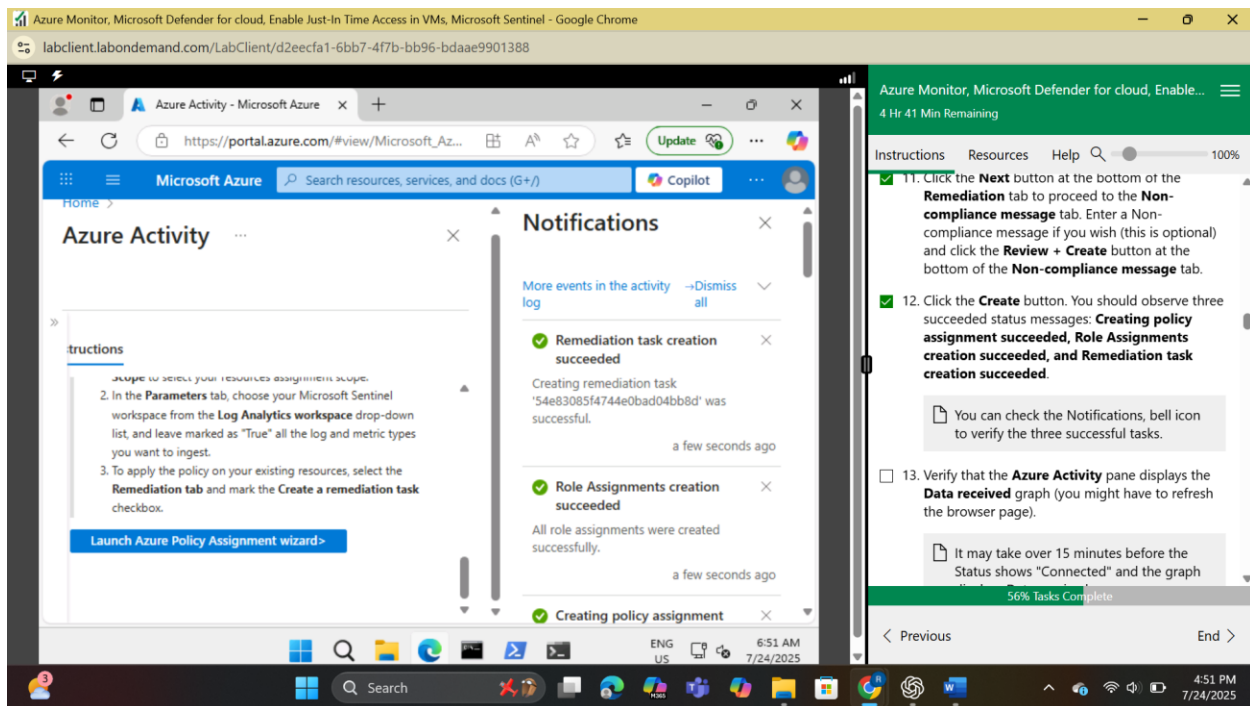


Fig 3.9 Successfully creating the assignments

Exercise 3: Creating a rule that uses the Azure Activity data connector

Next, I created a rule using the Azure Activity data connector. From the Microsoft Sentinel configuration blade, I clicked on Analytics, selected the Rule templates tab, and searched for "Suspicious." I chose the rule titled "Suspicious number of resource creation or deployment," which is associated with Azure Activity, and clicked Create rule. I accepted all the default settings in the General, Set rule logic, Incident settings, and Automated response tabs. On the final Review and create tab, I clicked Save, activating the rule.

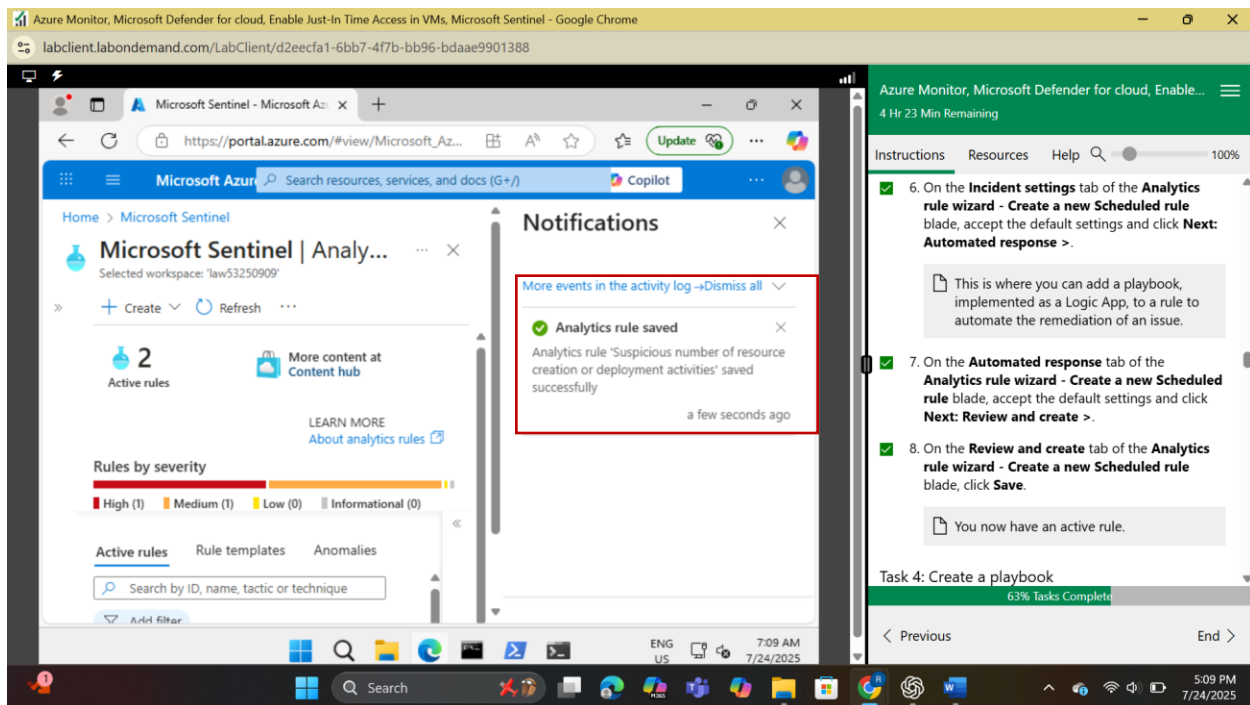


Fig 4.0 Successfully creating a new scheduled rule

Task 4: Creating a playbook

To automate responses, I created a playbook. In the Azure portal, I searched for "Deploy a custom template" and selected "Build your own template in the editor." In the template editor, I clicked Load file and uploaded the "changeincidentseverity.json" file located in the provided lab path. After loading, I saved the template.

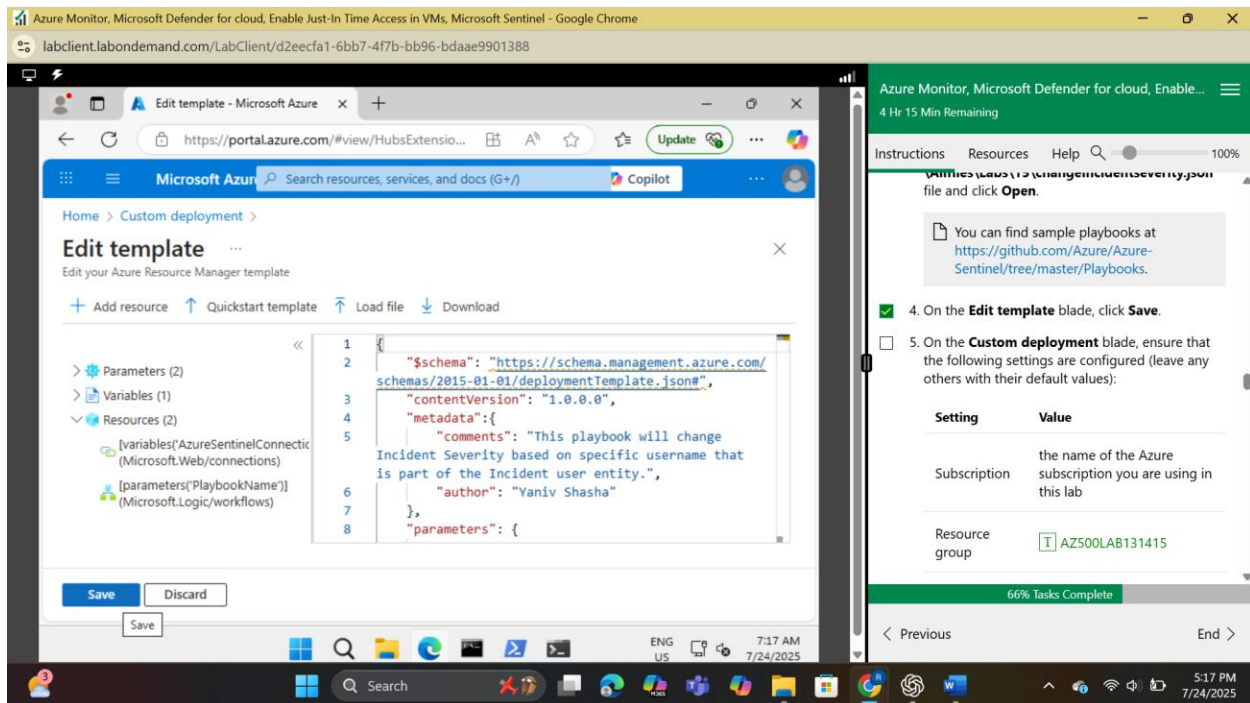


Fig 4.1 Editing json file

On the Custom deployment blade, I set the subscription to the one used for the lab, chose the resource group AZ500LAB131415, and set the location to East US. I named the playbook "Change-Incident-Severity" and entered my email address. I reviewed the details and clicked Create.

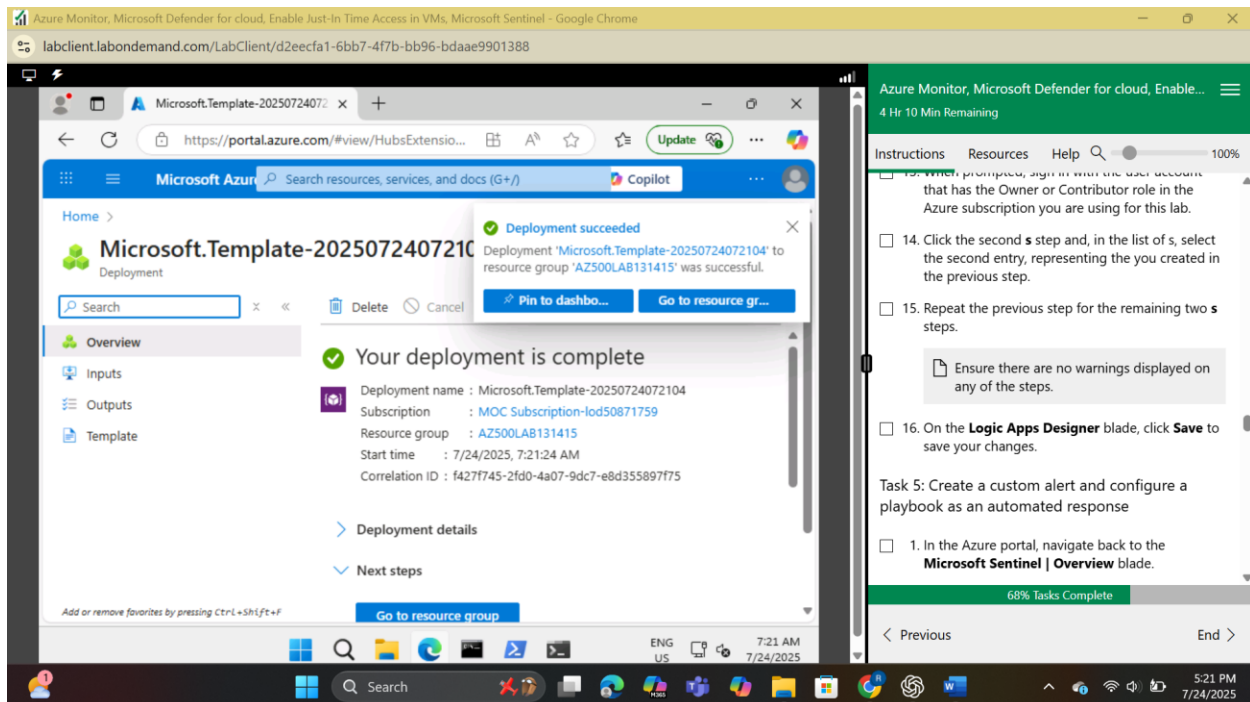


Fig 4.2 Successful deployment

After deployment, I navigated to the AZ500LAB131415 resource group and selected the Change-Incident-Severity logic app. On its blade, I clicked Edit. The Logic App Designer showed warning symbols for each step, indicating missing configurations. I clicked on the first step, added a new connection, verified my Azure AD tenant, and signed in. I repeated this process for the remaining three steps to resolve all warnings. Once all steps were configured correctly, I saved the logic app.

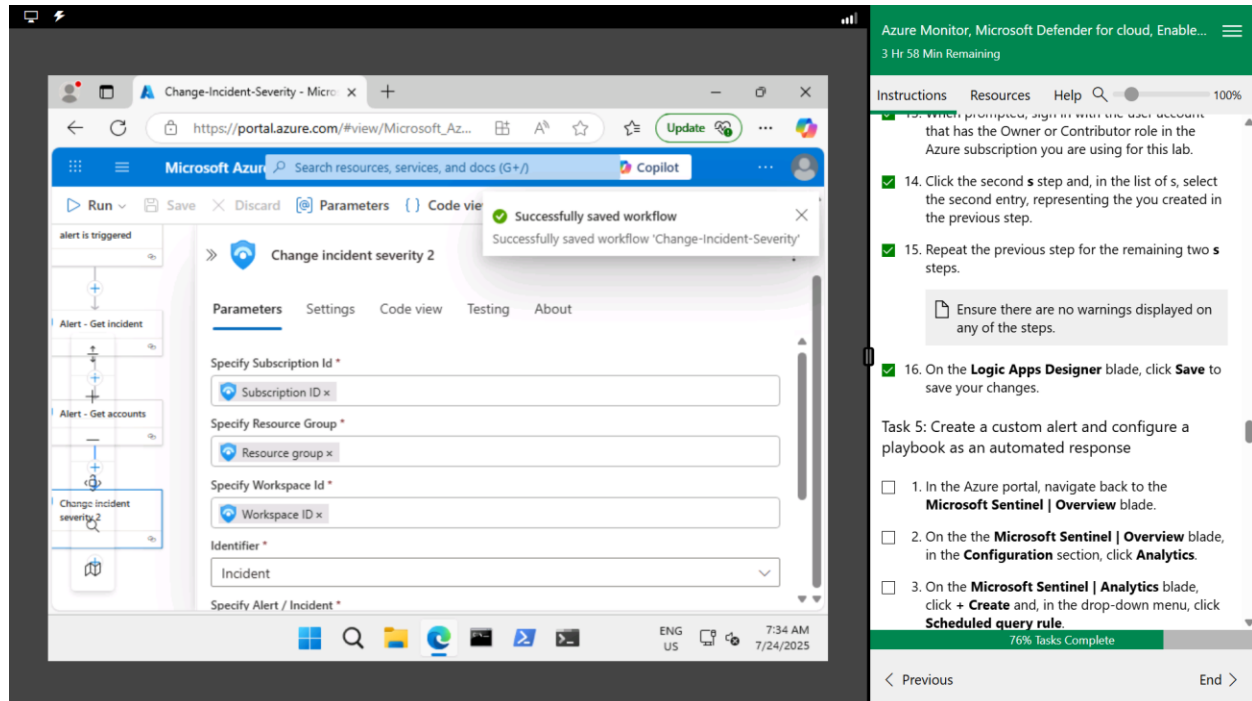


Fig 4.3 Successfully saving the workflow

Task 5: Creating a custom alert and configure a playbook as an automated response

Following this, I created a custom alert and linked the playbook as an automated response. I returned to the Microsoft Sentinel Overview blade, selected Analytics under Configuration, and clicked + Create, choosing Scheduled query rule. On the General tab, I named the rule "Playbook Demo" and selected "Initial Access" as the tactic. On the Set rule logic tab, I entered the following KQL query:

```
AzureActivity | where ResourceProviderValue =~ "Microsoft.Security" | where  
OperationNameValue =~ "Microsoft.Security/locations/jitNetworkAccessPolicies/delete"
```

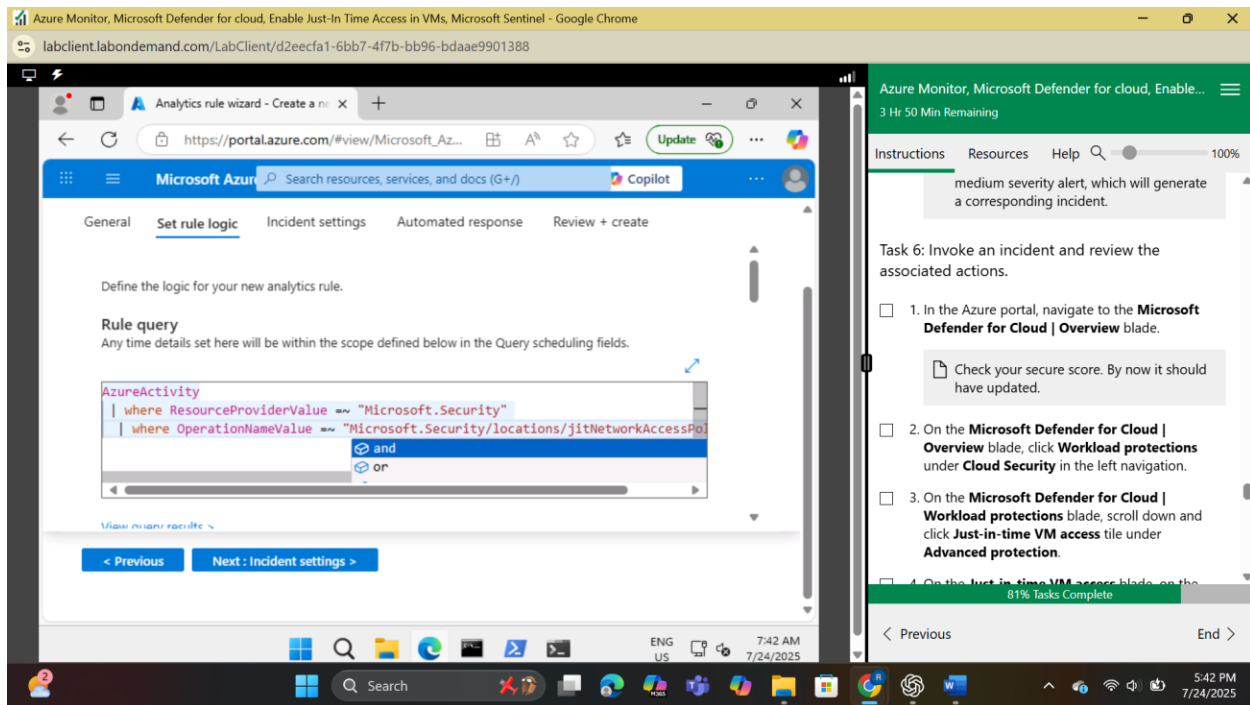


Fig 4.4 Entering KQL query on the set rule tab

This query detects deletion of Just-in-time VM access policies. I set the query to run every five minutes and kept all other default settings, clicking through to the Automated response tab. There, I added a new automation rule, named it "Run Change-Severity Playbook," and set the trigger to "When alert is created." I clicked "Manage playbook permissions," selected the AZ500LAB131415 resource group, and clicked Apply. I then selected the Change-Incident-Severity logic app and completed the setup by clicking Apply and then Save.

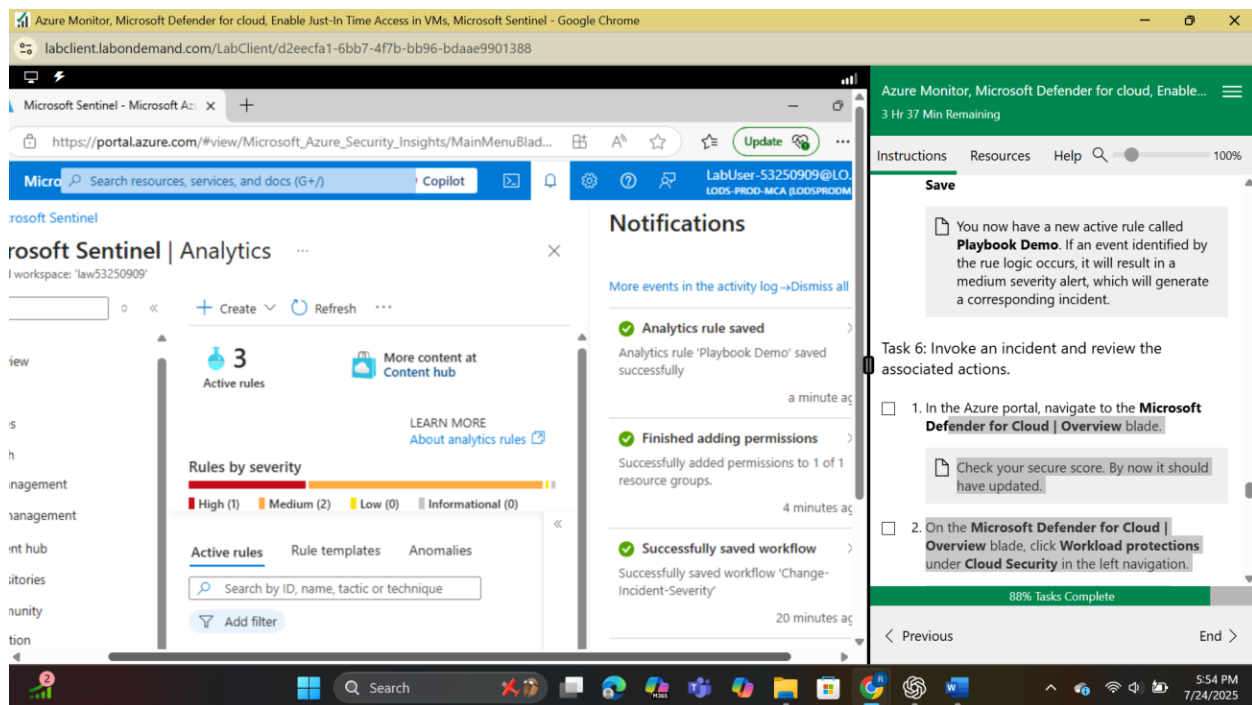


Fig 4.5 Successfully creating the analytics rule

Task 6: Invoke an incident and review the associated actions

To test the incident, I went to Microsoft Defender for Cloud and reviewed the secure score. Under Workload protections, I selected Just-in-time VM access. On the row corresponding to the "myVM" virtual machine, I clicked the ellipsis button, chose Remove, and confirmed the action.

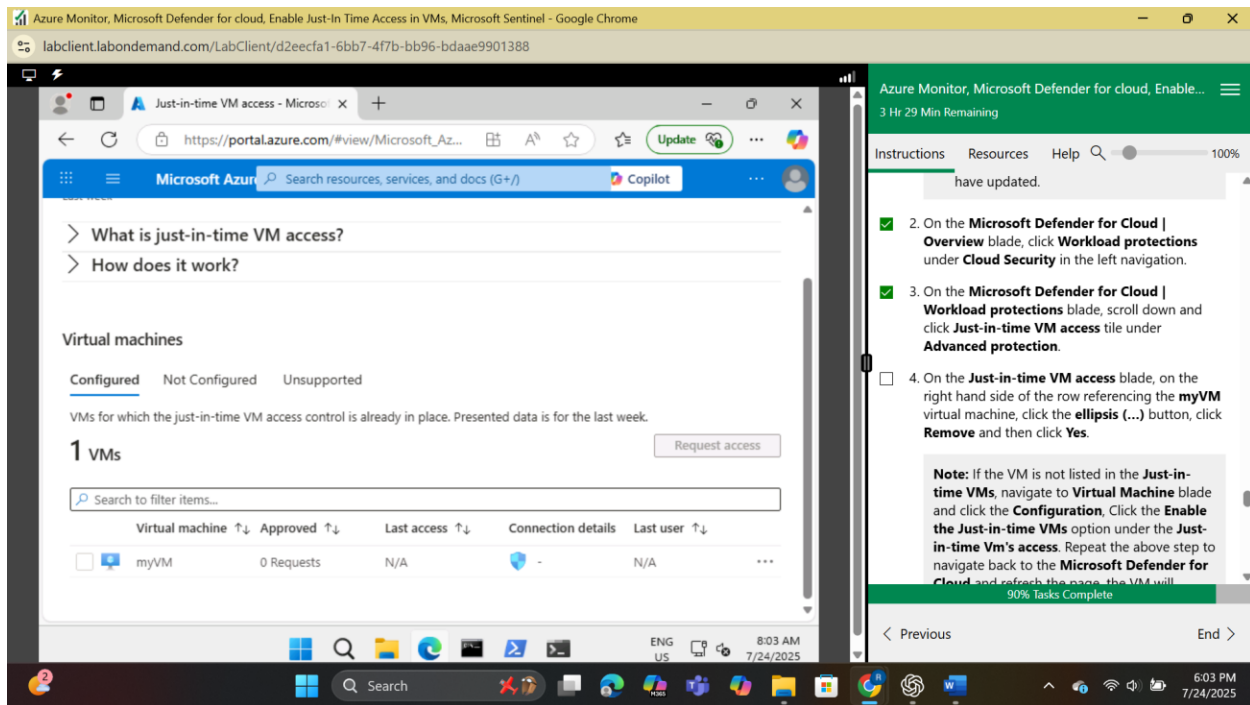


Fig 4.6 Removing myVM

I verified the action by checking the Activity log, where a “Delete JIT Network Access Policies” event appeared.

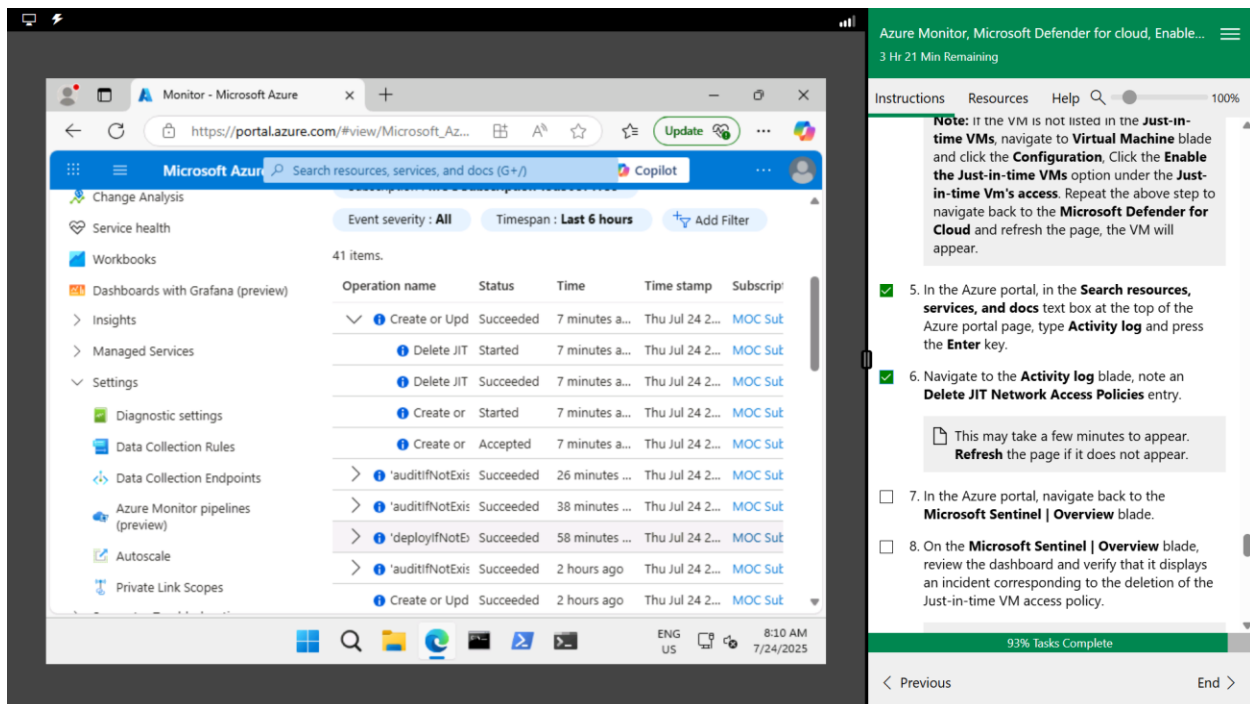


Fig 4.7 Checking the activity log

I then returned to the Microsoft Sentinel Overview blade. Within a few minutes, an incident appeared on the dashboard and under the Incidents section with a or high severity level. I also reviewed the Playbooks blade, which showed successful and failed executions, and confirmed that the triggered alert had invoked the playbook successfully.

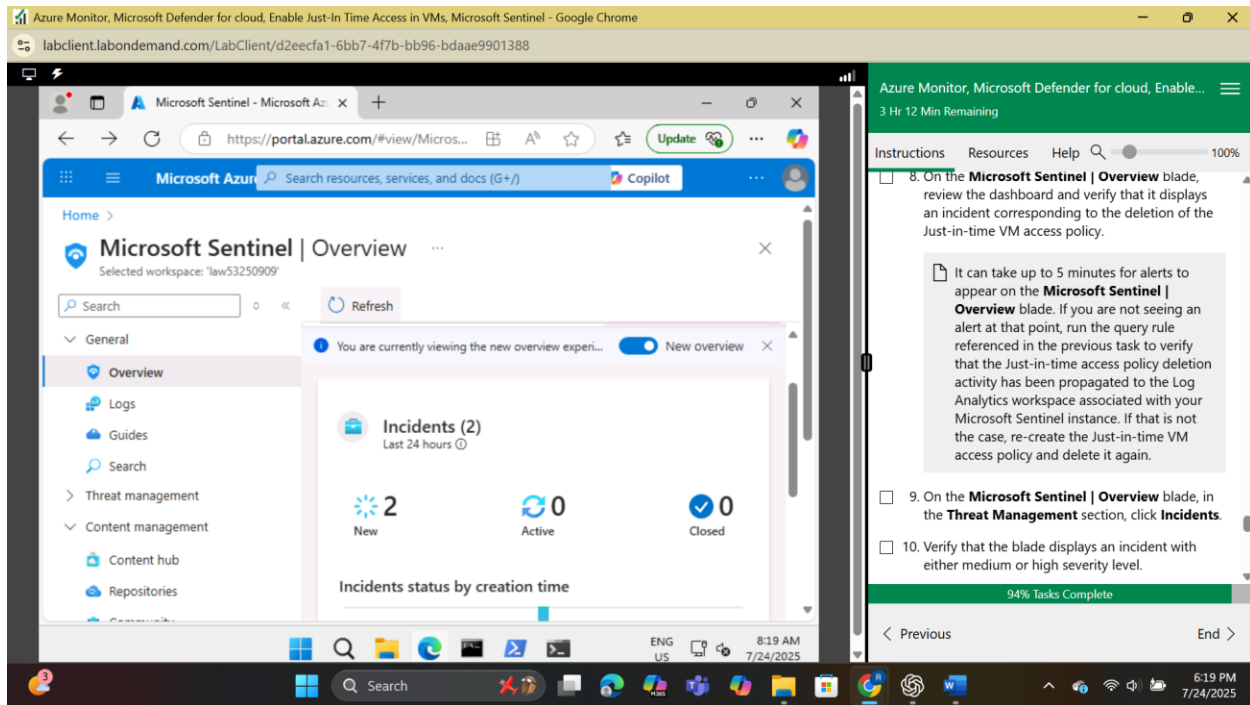


Fig 4.8 Microsoft Sentinel showing incidents with high level severity

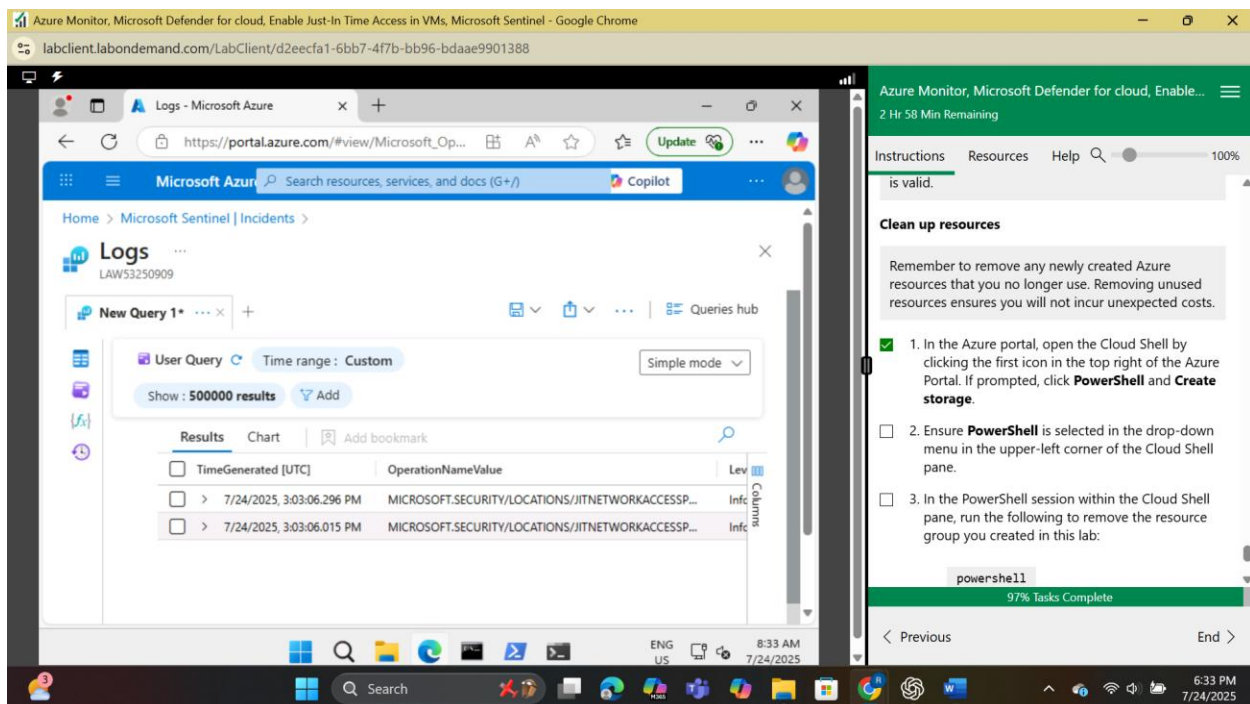


Fig 4.9 Analytic info on the deleted JIT myVM

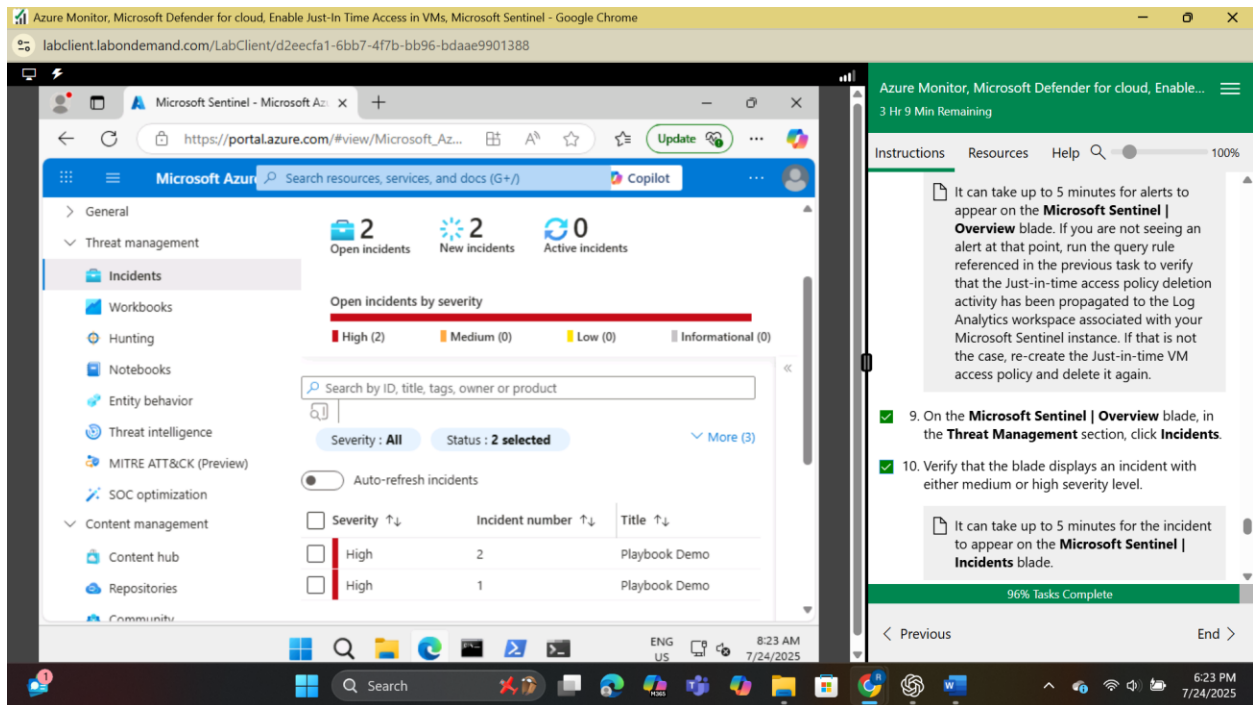


Fig 5.0 The incident section showing activities with high severity

Finally, I cleaned up the resources used in the lab. I opened the Cloud Shell in Azure, selected PowerShell, and entered the following command to delete the resource group:

```
Remove-AzResourceGroup -Name "AZ500LAB131415" -Force -AsJob
```

After confirming the job started, I closed the Cloud Shell pane.

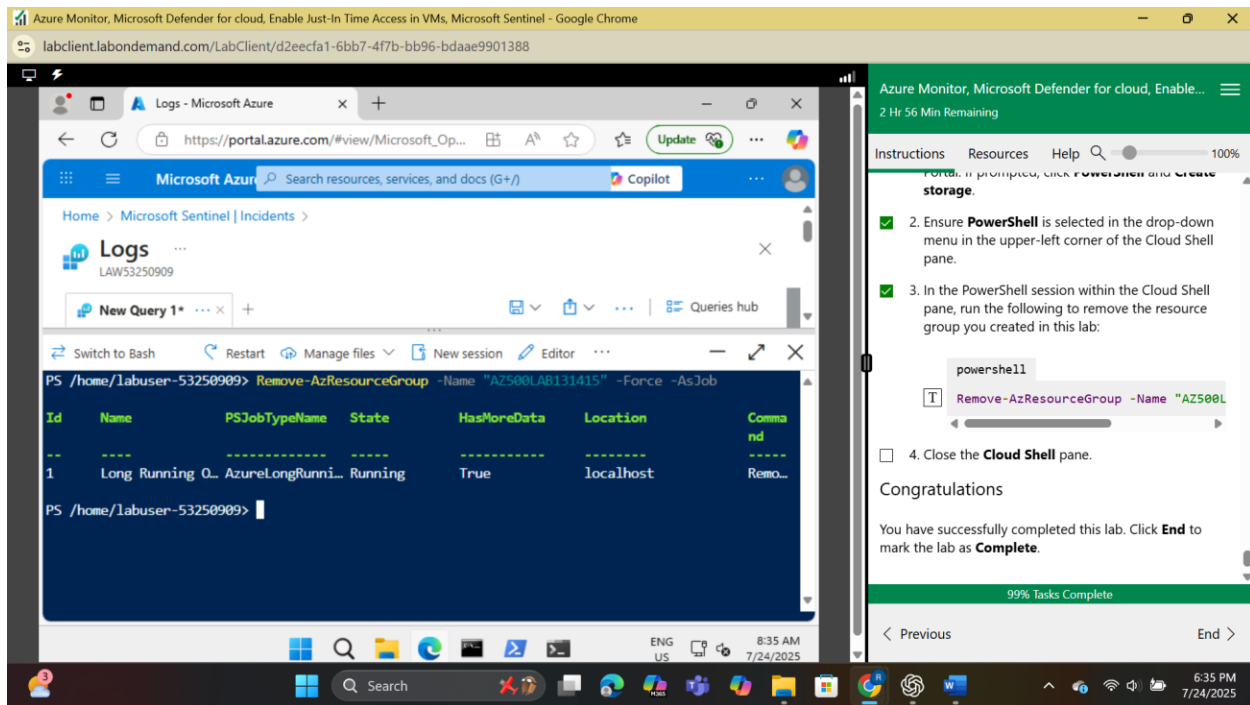


Fig 5.1 Cleaning up the resource

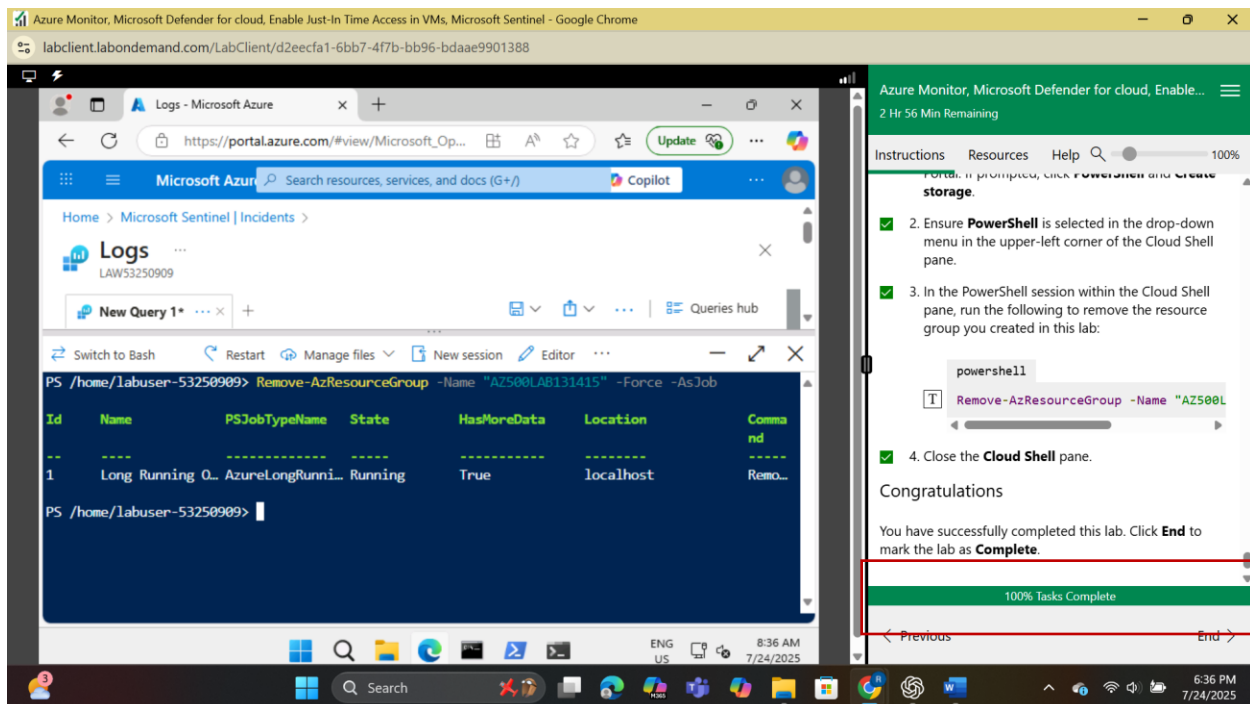


Fig 5.2 Lab completion

This concluded Lab 11. I successfully created a Microsoft Sentinel workspace, connected it to Azure Activity logs, configured both a playbook and custom alert to respond to Just-in-time VM access policy deletions, and validated the end-to-end response flow.

Conclusion

Throughout Labs 8 to 11, I applied a comprehensive set of security tools to protect and monitor Azure resources effectively. I deployed and secured a virtual machine, enabled advanced protection with Microsoft Defender for Servers Plan 2, configured just-in-time access to limit unnecessary exposure, and implemented Microsoft Sentinel to detect, respond to, and automate alerts based on real-time activity. These labs provided a well-rounded, practical approach to modern cloud security operations, reinforcing the importance of visibility, control, and automation in safeguarding cloud infrastructure.